

软考系统架构设计师

完整论文范文集

基于 H.I.S. BASE HOME 项目真实架构撰写

每篇含试题、摘要与正文，约 2800 字

共 10 篇

完整论文范文集 · 索引与使用说明

本目录包含 **9 篇可直接背诵的完整论文**，全部按软考「系统架构设计师」合格论文的格式撰写：标题 → 数字提醒 → 试题（含三问）→ 摘要 → 正文（五个小标题）→ 写作提示。

全部范文均基于 **H.I.S. BASE HOME 项目**（日本旅游业务系统）的真实架构撰写。

一、范文清单

文件	论题	正文核心难点素材	加粗亮点
01-论微服务架构的设计与应用.md	微服务架构的设计与应用	proto 三语言契约、依赖版本冲突、跨存储最终一致性	三个拆分维度（伸缩特征/访问协议/数据边界）
02-论云原生架构的设计与实现.md	云原生架构的设计与实现	事件驱动解耦、按业务域划分函数粒度、配置与凭证治理	主动写出云原生的 代价
03-论数据架构的设计与应用.md	数据架构的设计与应用	四类数据分类、DynamoDB 单表多实体、读写分离、多时区	四类数据分类法、单表多实体建模
04-论高可用架构的设计与实现.md	高可用架构的设计与实现	资源隔离、幂等重试、状态外置	「最大威胁来自内部资源争抢」
05-论信息系统安全架构的设计.md	信息系统安全架构的设计	默认拒绝的授权重构、临时凭证、会话安全	数据三级分类、「默认拒绝」决策
06-论系统集成方法及其应用.md	系统集成方法及其应用	proto 五层契约、三语言产物生成、异步集成幂等	「依赖版本也是一种隐式契约」
07-论软件架构评估方法及其应用.md	软件架构评估方法及其应用	ATAM 五阶段、效用树、六要素场景、敏感点/权衡点/风险点	4 个权衡点全部来自项目真实决策
08-论批处理系统的设计与实现.md	批处理系统的设计与实现	资源隔离四措施、参数化基准日、指标主动推送	「批处理的差异在可观测性模型」
09-论软件架构的演化与维护.md	软件架构的演化与维护	契约先行、渐进替换、防腐层、隐式契约显式化	「能退回比能修好更重要」

备考优先级建议

- 只求通过（45 分）：精背 **01、03、04、07**，这四类的题面覆盖率最高
- 求高分：再补 **06、09**，这两类题目竞争者少、容易出彩
- 保险起见：**02、05、08** 至少各读两遍，记住摘要与三个难点名称

二、九篇共用的一套数字（重要）

9 篇范文共用同一套项目口径，**保证互相引用时不矛盾**。替换时请对照本表统一修改，不要只改其中一篇——阅卷老师若发现同一项目在不同论文中数字冲突，会质疑真实性。

2.1 已从项目源码核实（可直接使用）

项目口径	真实值	出现于
关系型数据库表数	79 张（主数据表 26 张 m_ 前缀 + 业务表 53 张 t_ 前缀）	01、03
云数据库表数	5 张	01、03
远程调用接口数	17 个（单一服务对外提供）	01、06
契约文件数	89 个 proto（服务定义 1 + 请求 17 + 响应 17 + 枚举 12 + 可复用消息 42）	01、06
无服务器函数数	14 个（6 个队列触发 + 5 个数据流触发 + 3 个定时触发）	02、04、07
批处理作业数	10 个	08
批量游标 / 扫描上限	200 条 / 10000	03、04、08
在线侧游标	1000 条	08
重试策略	最大尝试 4 次、间隔 1 秒	02、04、06、08
会话超时	28800 秒（8 小时，stage / qa / production）	04、05
指标推送间隔	5 秒（Prometheus PushGateway，关闭动作 post）	04、08
错误码数量	API 侧 77 个（以消息编号统一定义）	06
调度链路	EventBridge Scheduler（cron，Asia/Tokyo）→ Step Functions → AWS Batch（ECS Fargate）	08
遗留 SDK 范围	对象存储 / 安全令牌服务两项	09

2.2 项目中不存在，必须由你替换

项目口径	示例值	出现于
项目启动 / 上线时间	2024 年 3 月 / 2025 年 1 月	全部 9 篇
营业门店数	300 余家	全部 9 篇
业务人员数	约 5000 名	全部 9 篇
注册旅行者数	800 万名	01
日均预约量	约 3 万笔	01、02、04、06、07、08、09
日均消息与通知量	约 12 万条	01、02、04、07、08
年可用率	99.95%	01、02、04、07、09

项目口径	示例值	出现于
写入主链路响应时间	约 1.8 秒 → 420 毫秒	01、02、07
批量作业耗时下降	约 30%	03、04、07、08
平峰期计算成本下降	约 35%	02、07
接口一致性缺陷下降	约 85%	01、09
消息重复发送率	0.3% → 0.01%	02、04、06
稳定运行时长	约 8 个月	全部 9 篇

⚠ 2.2 全部为**示例值**——门店数、用户数、预约量这类业务规模指标，在项目资料中**尚未确定**（性能测试设计书对应单元格标注为「数値の検討」，即数值待定），因此无法从源码查证，必须由你替换。每篇范文末尾另有该篇专属的「待替换数字清单」，两处需保持一致。

2.3 项目中真实存在的性能指标（可放心引用）

业务规模数字虽未确定，但**性能指标是真实存在的**，且来自正式的测试设计书与实测日志。用它们替换范文中的性能数字，比编造更有说服力，也经得起追问。

指标	真实值	出处
全事务 90 分位响应时间	5 秒以内	综合测试（性能）シナリオ一覧
部分场景响应时间	场景 1・3：2 秒；场景 2・4：1 秒（90% 以上达标）	同上
批处理作业时长	5 小时以内（实测 4 小时 47 分 ）	同上
批处理吞吐量	10 tps（实测达成）	同上
批量处理记录数	单批 2 万件超	批处理实测日志
单次批量作业实测耗时	111504 毫秒（约 1 分 51 秒 ）	<code>base-home-batch</code> StopWatch 日志
门店维度响应时间样例	1527 毫秒（90 百分位，店舗番号 B40）	性能测试结果

出处：`design-book\BASE_HOME\10_仕様書\30_試験\510_性能\総合テスト（性能）シナリオ一覧・シナリオ明細_BASE-HOME.xlsx`

替换建议：把范文中的"改造后响应时间 320 毫秒"改为与本项目口径一致的 "**90 分位响应时间控制在 5 秒以内**";把"批量作业耗时下降约 30%"改为 "**批处理作业在 4 小时 47 分内完成，满足 5 小时以内的目标，吞吐量达到 10 tps**"。这两组数字有测试报告支撑，是九篇范文中**最经得起追问的量化素材**。

三、使用方式

方式一：直接背诵（推荐）

1. 从本目录的 `PDF/` 中打印你选定的 3~4 篇

- 2. 先背**摘要**（约 330 字）——摘要决定阅卷老师的第一印象
- 3. 再背**正文的五个小标题**——这是文章的骨架，考场上先写骨架再填内容
- 4. 最后背每篇结尾的「可复用要点」——这是该题型的得分关键句

方式二：改造后背诵

每篇末尾的「写作提示」列出了：

- **待替换数字清单** —— 把示例值换成你的真实数据
- **可复用要点** —— 提示哪些句子是"通用高分句"，可跨题目迁移
- **本文字数** —— 核对是否在 2500 ~ 3000 字区间

方式三：考场应急组合

若考题的问法不在上述 9 篇之内，按下列方式组合：

考题关键词	主用范文	可插入的段落
性能 / 响应时间	04 高可用	03 的读写分离、02 的事件驱动解耦
可维护性 / 可扩展性	09 演化	01 的契约单一来源、06 的统一版本管理
可靠性 / 容错	04 高可用	08 的幂等与重跑、02 的部分失败处理
数据处理 / 存储选型	03 数据架构	08 的游标限制与扫描边界
分布式 / 跨系统协作	06 系统集成	01 的契约生成链路、02 的配置治理
技术选型 / 方案对比	07 架构评估	03 的选型依据、02 的容器与函数取舍

四、必须守住的写作红线

4.1 项目中并不存在，不要编造

- Kubernetes / Istio / 服务网格 / 服务注册中心（Eureka / Nacos / Consul）
- 熔断器、限流组件（Sentinel / Hystrix / Resilience4j）
- 分库分表、ShardingSphere、读写分离**中间件**（注：项目有读写数据源隔离，但那是代码层的数据源分离，不是数据库中间件，二者不可混为一谈）
- 多活 / 异地多活 / 双活数据中心
- Redis 作为业务缓存（注：项目中 ElastiCache Serverless（Valkey）**仅用于会话存储**，不承载业务缓存，范文已按此口径表述）
- 堡垒机、入侵检测、国密算法、等级保护认证

4.2 项目中真实存在，可放心引用

以下技术均已在本项目落地，若考题涉及相关领域可以直接写，不必担心被追问：

领域	真实技术
计算	ECS Fargate、AWS Batch、Lambda、EventBridge Scheduler、Step Functions

领域	真实技术
存储	Aurora MySQL、DynamoDB（含 Stream）、S3
消息	SQS（FIFO）、SNS
安全	AWS WAFv2、Cognito（OIDC）、STS 临时凭证、Systems Manager Parameter Store
会话	ElastiCache Serverless（Valkey）
框架	Java 21、Spring Boot 3.5.7、Spring Batch、MyBatis、gRPC / Protobuf、Spring Retry
构建	Gradle 8.14.3、统一版本管理插件
可观测	Prometheus PushGateway、结构化日志、链路追踪
测试	TestContainers、LocalStack、Pytest
基础设施	CloudFormation（基础设施即代码）

WAF 的说明：早期版本的本表曾把 WAF 列为"不可编造"，这是**错误的**——项目确实通过 CloudFormation 定义了 AWS WAFv2。已更正。

阅卷老师中不乏一线架构师。编造自己没用过的技术，一旦被追问细节就会露馅；而"主动写出方案的不足与未采用的方案"，反而是有架构判断力的表现。这一点在 04、05 两篇的结尾段有示范。

范文内容随项目架构变化需要更新时，修改对应的 `.md` 文件后，在 `_tools/` 目录运行 `python md2pdf.py` 重新生成即可。

论微服务架构的设计与应用

⚠ 关于本文中的数字 本文的量化数据（业务量、响应时间、可用率、成本等）均为**示例值**，用于展示论文应有的完整度与写法。**架构类数字**（表数量、接口数、配置值）已从项目源码核实，可直接使用；**业务规模类数字**（门店数、用户数、业务量、改善幅度）在项目资料中尚未确定，属示例值，背诵前请替换为你的**真实数据**。文末附有「待替换数字清单」，便于逐项核对。

试题

论微服务架构的设计与应用

微服务架构是一种将单一应用程序划分为一组小型服务的架构风格，每个服务运行在独立的进程中，服务之间通过轻量级的通信机制相互协作。微服务架构具有独立部署、技术异构、按需扩展等特点，能够有效应对业务复杂度的增长和快速交付的要求，但同时也带来了服务拆分、分布式数据一致性、服务治理等方面的新挑战。

请围绕"微服务架构的设计与应用"论题，依次从以下三个方面进行论述：

- 1. 简要叙述你参与设计和开发的采用微服务架构的信息系统，以及你所担任的工作；

- 2. 详细论述你在该系统中如何进行服务边界的划分，如何实现服务间的通信与协作；
- 3. 分析微服务架构在本项目应用中遇到的问题及解决措施，并说明该架构的优缺点。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台为例，论述了微服务架构在该项目中的设计与应用。该平台是日本 H.I.S. 集团面向营业本部与旅行者建设的业务支撑系统，覆盖预约管理、客户消息、通知发送与主数据联动等核心业务，于 2024 年 3 月启动建设，2025 年 1 月上线运行。我在项目中担任系统架构师，负责服务边界划分、服务间通信契约设计与构建治理体系建设。针对业务伸缩特征差异大、跨语言调用契约易漂移、新旧技术栈需长期并存三大挑战，我按伸缩特征、访问协议与数据边界三个维度将系统划分为 Web、Service、Batch、FaaS 四类独立部署单元；以 Protocol Buffers 作为唯一契约源，驱动 Java、Go、PHP 三语言客户端生成；并通过统一版本管理插件把隐式依赖转为显式契约。系统上线后运行稳定，关键接口响应时间明显改善，接口一致性缺陷大幅下降，获得业务方认可。

正文

一、项目背景

随着日本旅游市场竞争的加剧和客户需求的日益个性化，H.I.S. 集团原有以人工受理为主的业务模式面临着越来越大的压力。营业本部在受理预约时需要在多个系统之间反复切换，客户消息与通知的发送依赖人工触发，不仅效率低下，而且容易出错。为此，集团决定建设 BASE HOME 平台，统一承载预约管理、客户消息、通知发送、文件管理与主数据联动等核心业务。

该平台于 2024 年 3 月启动建设，2025 年 1 月正式上线运行，服务集团 300 余家营业门店、约 5000 名业务人员以及累计 800 万名注册旅行者，日均处理预约约 3 万笔、各类通知与消息约 12 万条。系统基于 Java 21 与 Spring Boot 3.5.7 构建，部署于 AWS 东京区域。

我在项目中担任系统架构师，主要负责三个方面的工作：一是服务边界的划分与整体架构设计；二是服务间通信契约的设计与治理；三是构建体系与版本治理机制的建立。

项目启动时，我们面临着三个突出的架构挑战。第一，平台承载的业务在伸缩特征上差异极大：预约受理是随营业时间波动的交互型负载，成本文件生成是夜间的计算密集型批量负载，而数据同步则是持续低频的事件型负载。如果采用单体部署，任何一种负载的峰值都会波及其他业务。第二，平台需要同时为 Java 内部服务、Go 周边平台与 PHP 既有系统三类调用方提供服务，接口定义一旦分散维护，字段口径必然随时间漂移，而这类问题往往在上线之后才暴露。第三，平台需要长期兼容基于旧版 AWS SDK 的既有代码，无法通过一次性重写来完成技术栈统一。下面分别论述我针对这三个挑战所做的设计与实现。

二、服务边界的划分与总体架构设计

服务边界的划分是微服务架构设计中最为关键的环节。我在划分时没有采用“按技术分层拆分”这种常见做法——因为按控制层、业务层、数据层拆分会导致一次业务请求穿越多个服务，反而放大调用开销与故障面——而是确立了三个划分维度。

第一，按伸缩特征拆分。我梳理了平台全部业务负载的伸缩曲线，将其归纳为三类：交互型负载（营业本部员工的预约受理与查询操作，呈工作日昼间高峰）、计算型负载（成本文件生成、邮件批量生成等定时作业，呈夜间定时峰值且瞬时资源占用大）、事件型负载（DynamoDB 数据变更触发的同步处理，持续低频）。据此，我把

系统拆分为 Web、Service、Batch、FaaS 四类独立部署单元，使每一类都能按自身的伸缩曲线独立扩缩容。这一决策牺牲了单体部署时进程内调用的简单性，换来的是资源隔离与独立伸缩能力——在旅游业务的强季节性场景下，这个交换是值得的。

第二，按访问协议与调用方拆分。内部服务之间的调用我选择了 gRPC 而非 REST，主要基于三点考虑：gRPC 基于 HTTP/2 多路复用，在高频内部调用场景下连接利用率更高；采用二进制编码，序列化开销与报文体积更小；更重要的是，protobuf 天然提供强类型契约，这与我们后文将要论述的契约治理策略高度契合。对内对外则通过 Web 层暴露 REST 接口，兼顾浏览器友好性与前后端契约的稳定性。

第三，按数据存储特征拆分。这是我在此次设计中最为重视的一点。我采用混合存储策略：Aurora MySQL 承载需要事务与复杂关联查询的强一致交易数据，共 79 张表，其中主数据表 26 张（m_ 前缀）、业务表 53 张（t_ 前缀）；DynamoDB 承载字段演进频繁的半结构化数据，共 5 张领域表；S3 承载护照、文档、亲权同意书等文件实体。这样做的核心目的，是让服务边界与数据边界对齐——如果多个服务共享同一张表，那么无论代码上如何拆分，它们在数据层面依然是耦合的，任何一次表结构变更都会形成跨团队的连锁影响。

最终形成的架构是：Web 层承接交互请求，通过 gRPC 调用 Service 层的业务能力；Service 层访问 DynamoDB 与 S3；Batch 层承担夜间批量作业并通过独立数据源访问 MySQL；FaaS 层由 DynamoDB Stream 与消息队列驱动，负责跨存储的数据同步。

三、关键技术实现

契约驱动的服务通信。针对多技术栈调用方的接口一致性问题，我在独立的规格模块中确立了以 protobuf 作为唯一契约源的策略。平台全部服务接口以 89 个 proto 文件定义，按职责分为服务定义、请求消息、响应消息、枚举与可复用消息类型五层组织——其中服务定义 1 个、请求与响应消息各 17 个、枚举 12 个、可复用消息 42 个。可复用消息层沉淀了错误信息、地址、旅行者、护照等被多个接口引用的公共结构，避免了同一结构在十几个接口中重复定义。在此基础上，我通过构建脚本编排了一条完整的生成链路：先由 protobuf 插件生成 Java 产物并发布至内部 Maven 仓库；再在容器化的 protoc 环境中生成 Go 产物并额外生成网关代码，发布为 Go 模块；随后生成 PHP 产物发布为 Composer 包；最后由 protoc 生成 OpenAPI 描述文件，经文档站点发布为在线接口文档。平台以单一服务对外提供 17 个远程调用接口，每个接口在契约中显式声明了统一的 HTTP 映射路径，三端口径由同一份契约保证一致。

服务端的统一分发设计。服务端以单一入口类承接全部远程调用，未在每个接口上重复编写样板代码，而是通过统一的执行方法将请求分发到各业务实现类。这样设计的价值在于横切关注点的集中：日志记录、异常转换、调用上下文校验都可在分发层统一处理，各业务实现类只需关注业务逻辑本身。同时，我把被多个业务域共用的能力——文档、消息线程、亲权同意书、序列号、旅行信息——下沉为共享服务，避免同一逻辑在多个接口中重复实现。当然，单一入口在接口数量极大时会成为维护瓶颈，但以平台当前的接口规模而言，统一分发带来的收益明显大于其复杂度成本。

数据访问与批量处理。关系型数据库侧统一采用 MyBatis 作为持久层框架，并纳入统一版本管理；NoSQL 侧通过增强客户端提供对象映射，通用操作封装在仓储基类中。批量作业方面，我为批处理配置了独立的读写数据源，通过包路径隔离数据访问接口：读取数据源指向 Aurora 的 cluster-ro 只读副本端点，写入数据源指向 cluster 主库端点，避免夜间批处理峰值挤占在线业务的连接资源。

四、实施中遇到的问题及解决措施

问题一：跨语言接口契约的一致性维护。平台需同时服务 Java、Go、PHP 三类调用方，项目初期各端曾各自维护接口定义，导致同一字段在不同端的命名与类型出现差异，联调阶段才被发现。我的判断是，这类问题的根源

不是开发人员疏忽，而是**契约缺乏单一来源**这一架构缺陷。因此我没有停留在逐处修正，而是把接口定义全部收敛到规格模块，使接口变更必须通过该模块提交；同时把三语言产物的生成纳入构建流水线，让接口不一致问题从"运行时联调发现"提前到"构建期暴露"。改造后，接口层面的联调返工显著减少。

问题二：依赖版本漂移引发的运行时故障。 平台初期各模块独立声明依赖版本，部分组件采用了动态版本区间。一次依赖升级后，protobuf 官方公共类型库被解析到一个候选版本，导致 gRPC 服务启动直接失败。这次故障让我意识到，我们虽然统一了业务接口的契约，却忽略了**第三方依赖版本这一隐式契约**。因此我建立了独立的统一版本管理插件，各模块通过插件接入统一的版本源，并对关键组件使用强制版本策略加以锁定。改造后，同一提交在任何机器上的构建产物完全一致，构建的可复现性得到保障。

问题三：跨存储的数据最终一致性。 混合存储带来了新的问题：写入主链路落在 DynamoDB，而报表与既有系统仍需从 MySQL 读取数据。若在服务端同步写两侧，主链路响应时间会被显著拉长。我的方案是把同步链路设计为事件驱动的无服务器架构：以 DynamoDB 数据流与消息队列作为事件源，驱动按业务域划分的十四个函数完成同步，主链路只写 DynamoDB 即返回。这一设计以最终一致性替代了强一致性——经评估，报表与通知场景可以接受秒级延迟，而预约写入等强一致场景仍走关系库事务，不做异步化。该方案使主链路响应时间由约 1.8 秒降至 420 毫秒。

五、总结与展望

平台上线至今已稳定运行约 8 个月，年可用率达到 99.95%，支撑业务量较原系统提升约 40%，接口一致性缺陷较改造前下降约 85%。回顾整个建设过程，我认为微服务架构在本项目的价值主要体现在三个方面：一是使业务特征差异极大的模块得以独立演进，批处理的资源峰值不再影响在线预约受理；二是通过契约的单一来源，让多技术栈协作从"人工对齐"变为"构建期保证"；三是使新增业务可以自然地落在最合适的存储与部署形态上。

当然，这一架构也带来了明显的代价。首先是分布式调试与问题定位的复杂度上升，一次跨服务请求的排查需要串联多个组件的日志；其次是跨服务的事务能力缺失，我们只能通过最终一致性来弥补；再次是运维单元增多，环境治理与监控的成本高于单体架构。

目前仍存在两点不足：一是跨语言契约虽有统一来源，但破坏性变更尚未实现自动化的兼容性检查，仍依赖人工评审；二是旧版云 SDK 与新版并存的过渡状态尚未终结，长期维护成本依然存在。后续计划引入契约兼容性校验工具，并逐步收敛 SDK 版本。

通过本项目，我深刻体会到微服务架构的价值并不在于"拆"本身，而在于让每个服务能够按照自身的业务节奏独立演进。架构师在面对微服务时，真正需要回答的问题不是"要不要拆"，而是"拆开之后，那些原本由进程内调用免费获得的确定性——事务、强类型、可调试性——要用什么来补回来"。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

已从源码核实，可直接使用

位置	真实值
契约文件数 / 远程接口数	89 个 proto / 17 个接口
契约分层构成	服务定义 1 + 请求 17 + 响应 17 + 枚举 12 + 可复用消息 42
关系库表数 / 云库表数	79 张 (m_26 + t_53) / 5 张

位置	真实值
无服务器函数数	14 个（6 队列 + 5 数据流 + 3 定时）

项目中不存在，需你替换

位置	示例值	请替换为
项目启动与上线时间	2024 年 3 月 / 2025 年 1 月	真实时间
营业门店数	300 余家	真实数量
业务人员数	约 5000 名	真实数量
注册旅行者	800 万名	真实数量
日均预约量	约 3 万笔	真实数量
日均消息量	约 12 万条	真实数量
稳定运行时长	约 8 个月	真实时长
年可用率	99.95%	真实指标
业务量提升	约 40%	真实数据
接口缺陷下降	约 85%	真实数据
主链路响应时间	1.8 秒 → 420 毫秒	建议改用「90 分位 5 秒以内」（有测试报告支撑）

本文字数

摘要约 320 字，正文约 2700 字，**合计约 3020 字**（软考要求 2500 ~ 3000 字）。

可复用要点

- **三个拆分维度**（伸缩特征 / 访问协议 / 数据边界）是本文的骨架，可直接迁移到其他项目
- **"契约的单一来源"** 与 **"隐式契约显式化"** 是两个有分量的架构判断，适合用于"可维护性""架构演化"类题目
- 若考题侧重"服务治理"，把第三节展开；若侧重"拆分"，把第二节展开；若侧重"挑战"，把第四节展开

论云原生架构的设计与实现

⚠ 关于本文中的数字 本文的量化数据（业务量、响应时间、成本降幅等）均为**示例值**，用于展示论文应有的完整度与写法。架构类数字（表数量、接口数、配置值）已从项目源码核实，可直接使用；业务规模类数字（门店数、用户数、业务量、改善幅度）在项目资料中尚未确定，属示例值，背诵前请替换为你的真实数据。文末附有「待替换数字清单」，便于逐项核对。

试 题

论云原生架构的设计与实现

云原生架构是基于云原生技术的一组架构原则和设计模式的集合，旨在将云应用中的非业务代码部分进行最大化的剥离，从而让云设施接管应用中原有的大量非功能特性，使业务不再有非功能性业务中断困扰的同时，具备轻量、敏捷、高度自动化的特点。云原生架构常用的技术包括容器、服务网格、微服务、无服务器计算等。

请围绕“云原生架构的设计与实现”论题，依次从以下三个方面进行论述：

- 1. 简要叙述你参与设计和开发的采用云原生架构的信息系统，以及你所担任的工作；
- 2. 详细论述你在该系统中如何运用云原生技术进行架构设计，解决了哪些实际问题；
- 3. 分析云原生架构在本项目应用中存在的不足，并说明你的改进思路。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台为例，论述了云原生架构在该项目中的设计与实现。该平台部署于 AWS 东京区域，承载预约管理、客户消息、通知发送与主数据联动等核心业务。我在项目中担任系统架构师，负责容器化方案设计、无服务器数据同步链路设计与交付流水线建设。针对数据同步与写入主链路相互耦合、计算单元粒度难以把握、云上配置与凭证治理复杂三大挑战，我从事件驱动解耦、按业务域划分计算单元、配置与凭证集中治理三个维度进行设计：以数据流与消息队列作为事件源，驱动按业务域划分的十四个函数完成跨存储同步，使写入主链路只需落到云数据库即可返回，达成秒级的最终一致性；每个函数拥有独立镜像、独立流水线与独立部署单元，实现故障域隔离。系统上线后运行稳定，写入主链路响应时间由约 1.8 秒降至 420 毫秒，平峰期计算成本下降约 35%，验证了方案的可行性。

正文

一、项目背景

H.I.S. 集团是日本大型旅游企业，其业务具有极强的季节性特征：长假前后预约量激增，平峰期则迅速回落。这种“峰谷差极大”的负载特征，使传统的固定容量部署方式陷入两难——若按旺季峰值配置资源，全年大部分时间资源闲置；若按平峰配置，旺季则无法支撑。

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务集团 300 余家营业门店与约 5000 名业务人员，日均处理预约约 3 万笔、消息与通知约 12 万条。平台基于 Java 21 与 Spring Boot 3.5.7 构建，部署于 AWS 东京区域。

我在项目中担任系统架构师，主要负责三个方面的工作：一是容器化方案与运行基线设计；二是无服务器数据同步链路的设计；三是配置与凭证治理机制以及交付流水线的建设。

项目面临的三个核心挑战是：第一，平台采用混合存储，写入主链路落在云数据库（NoSQL），而报表与既有系统仍需从关系库读取数据，若在服务端同步写两侧，主链路响应时间将被显著拉长；第二，同步逻辑若作为一个整体部署，其扩容需求会绑架整个服务的资源配置，且一个业务域的故障会波及全部同步任务；第三，云上配置与访问凭证若散落在代码与配置文件中，既存在泄露风险，也会因环境差异导致“本地能跑、线上报错”。下面论述我针对这三点所做的的设计。

二、云原生架构的总体设计

我把平台的云原生改造归纳为三个层次，自下而上依次是运行基座、计算形态与治理机制。

第一层是运行基座，即容器化交付。平台各模块分别构建独立镜像，统一基于集团提供的 Java 21 基础镜像，保证运行时基线一致；批处理模块不启用 Web 容器，仅作为短生命周期的计算进程运行，不占用端口，与在线业务在进程级完全隔离。

第二层是计算形态，即按负载特征选择容器或无服务器。我判断的标准是：负载是否持续、是否无状态、是否由事件驱动。在线交互与夜间批处理属于持续或定时负载，且需要稳定的运行环境，因此保留容器形态；而跨存储的数据同步属于持续低频、无状态、事件驱动的负载，非常适合无服务器形态，因此我将其设计为函数计算。

第三层是治理机制，包括配置管理、凭证管理与交付流水线。这一层最容易被忽视，却直接决定了云原生架构能否长期稳定运行。

无服务器同步链路的设计是本次改造的核心。我把同步链路拆解为四个部分：一是事件源，采用云数据库的数据流与消息队列，前者捕获数据变更事件，后者承接业务侧主动投递的消息；二是计算单元，即按业务域划分的十四个函数；三是配置源，全部配置与凭据统一存放于云参数服务，函数运行时按需读取；四是可观测能力，借助运行时框架提供结构化日志与链路追踪，并对消息队列的批量消费提供部分失败处理语义。基于这一设计，写入主链路只需将数据写入云数据库即可返回，同步由事件异步接管，两个关注点被彻底解耦。

三、关键技术的实现

按业务域划分函数粒度。在划分计算单元时，我没有采用“一张数据表对应一个函数”的做法。理由是：同一业务域内的实体更新往往需要一起处理，例如旅行信息发生变更时，需要同步更新申请人与预约数据，若按表拆分，则一次业务变更会触发多个函数，函数之间还需要额外编排，反而增加了复杂度与不一致风险。因此我按业务域划分，使每个函数内部能够完成一次完整的业务处理。这一设计同时带来了故障域的隔离——某个业务域的处理异常不会波及其他业务域。目前十四个函数中，六个由消息队列触发，五个由数据流触发，三个由定时任务触发。

批量消费与部分失败处理。函数消费消息队列或数据流时，一个批次中往往包含多条记录。若其中一条处理失败就导致整批重试，会造成大量重复处理。我借助运行时框架的批量处理工具实现了部分失败语义：消息队列侧按消息分组处理，只把失败分组中的记录返回给队列重试，正常分组照常提交；数据流侧同样只重试失败的记录。队列另配置了死信队列承接多次重试仍失败的记录。这一机制显著降低了重复处理的概率。

配置与凭证的集中治理。我把全部配置与敏感凭据从代码和配置文件中剥离，统一存放在云参数服务中，函数运行时通过专用的客户端按需获取，代码仓库中不保留任何明文凭据。服务端侧同样遵循这一原则：不使用长期固定的访问密钥，而是通过安全令牌服务获取临时凭证，并配合默认凭证链兜底，使凭证自动轮换且有效期极短。此外，我通过云数据库表名统一携带环境前缀（开发 / 测试 / 验收 / 生产四套环境各有独立前缀）的方式实现了环境隔离，避免测试环境误连生产数据。

独立交付流水线。每个函数拥有独立的镜像定义与独立的持续集成工作流，部署通过基础设施即代码工具完成。这带来的关键收益是：单个函数的变更不需要整体回归，支持单函数灰度与快速回滚。同时，容器化与本地云模拟工具的结合，使开发人员可以在本地拉起与云端一致的依赖服务（关系库用容器模拟、云数据库与对象存储用本地云模拟器模拟），消除了因环境差异导致的缺陷。

四、实施中遇到的问题及解决措施

问题一：同步链路的顺序性无法全局保证。数据流能够保证同一分区内的事件有序，但跨分区不保证顺序。若两个业务域的数据存在先后依赖，并行处理可能产生顺序错乱，导致旧数据覆盖新数据。我的应对措施是：把存在依赖关系的实体收敛到同一分区键之下，例如旅行相关的各类数据统一以旅行编号作为分区键，借由分区内的有

序性保证处理顺序；对于确实跨域的场景，则在数据中引入版本号，使乱序到达的更新可以通过比较版本号判定是否过期，从而避免覆盖。

问题二：消息推送的重复处理。 消息与邮件推送是本平台的可靠性敏感点，一旦重复发送，对客户而言是直接的体验事故，而朴素的失败重试必然会引入重复。我的方案分三层：第一层是以执行状态的幂等校验拒绝重复触发，从源头挡住重复；第二层是把重试能力独立为专用的重试服务，与业务逻辑解耦，使重试策略可独立调整而无需改动业务流程；第三层是投递状态跟踪表，逐条记录每一次投递的结果，使重跑时只处理未投递的记录，并在异常场景下可以精确定位漏发与重发的范围。改造后，重复发送率由改造前的约 0.3% 降至 0.01% 以下。

问题三：冷启动带来的延迟波动。 无服务器计算在低频函数上存在冷启动现象，首次调用延迟可达数秒。对于邮件生成等对延迟不敏感的场景，这一延迟可以接受；但对于由数据流触发的实时同步函数，则需要缓解。我采取的过渡措施是保留少量预热调用，并在后续规划中引入预置并发配置。需要说明的是，我并没有为此放弃无服务器方案——用函数形态换取弹性和按需计费，同时让部分延迟由业务侧的容忍度吸收，这个交换在本项目的负载特征下是合理的。

五、总结与展望

平台上线至今已稳定运行约 8 个月，年可用率达到 99.95%。云原生改造带来的收益主要体现在三个方面：一是写入主链路响应时间由约 1.8 秒降至 420 毫秒，用户体验明显改善；二是十四个函数按业务域独立扩缩容，任一函数故障不再波及其他业务域；三是按需计费的计价方式使平峰期计算成本下降约 35%，缓解了旅游淡旺季资源利用率失衡的问题。

同时，这一架构也带来了不可忽视的代价。首先是无服务器形态下的调试困难，本地难以完整复现云上的触发链路；其次是函数之间的依赖关系不像进程内调用那样直观，跨函数的数据流向需要额外维护文档；再次是链路追踪的复杂度上升，一次数据同步可能横跨数据库、事件源与多个函数，排查问题需要串联多方日志。

目前仍存在两点不足：一是跨存储的同步链路尚无统一的延迟监控，异常时的排查路径偏长；二是冷启动在低频函数上仍有可感知延迟。后续计划补充一致性延迟监控与预置并发配置，并逐步把函数间的依赖关系显式化。

通过本项目，我认识到云原生架构并非普适的解决方案。它真正适合的是事件驱动、无状态、负载突发型的场景；而对于强一致的事务处理、长时间运行的计算任务以及负载长期稳定的服务，传统的容器化部署仍是更稳妥的选择。架构师的职责，正是准确判断哪一部分负载属于前者、哪一部分属于后者。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

位置	示例值	请替换为
门店数 / 业务人员数	300 余家 / 约 5000 名	真实数量
日均预约量 / 消息量	约 3 万笔 / 约 12 万条	真实数量
函数数量与触发方式	14 个 (5 数据流 + 6 队列 + 3 定时)	✓ 已核实
主链路响应时间	1.8 秒 → 420 毫秒	真实压测数据
平峰期成本下降	约 35%	真实数据
重复发送率	0.3% → 0.01%	真实数据

位置	示例值	请替换为
稳定运行时长 / 可用率	约 8 个月 / 99.95%	真实指标
重试策略	最大 4 次 / 间隔 1 秒	✓ 已核实

本文字数

摘要约 330 字，正文约 2750 字，合计约 3080 字。

可复用要点

- "事件驱动解耦 + 最终一致性" 是本文骨架，凡涉及"高性能""高可用""数据一致性"的题目都可调用
- 按业务域而非按表划分函数粒度 是一个有说服力的架构判断，体现了"故障域隔离"思维
- 第四节主动写出云原生的代价（调试困难、依赖不直观、追踪复杂），是区别于普通考生的关键
- 若题目问"不适合云原生的场景"，直接引用结尾段的判断标准

论数据架构的设计与应用

⚠ 关于本文中的数字 本文的量化数据（数据量、查询延迟、作业耗时等）均为**示例值**，用于展示论文应有的完整度与写法。架构类数字（表数量、接口数、配置值）已从项目源码核实，可直接使用；业务规模类数字（门店数、用户数、业务量、改善幅度）在项目资料中尚未确定，属示例值，背诵前请替换为你的真实数据。文末附有「待替换数字清单」，便于逐项核对。

试题

论数据架构的设计与应用

数据架构是描述组织数据资产的结构、关系与管理规则的蓝图，它决定了数据如何被采集、存储、处理、共享和使用。良好的数据架构能够支撑业务的快速变化，保障数据的一致性与可用性，并为数据分析与决策提供基础。数据架构设计需要综合考虑数据特征、访问模式、一致性要求、扩展性需求以及成本约束等多方面因素。

请围绕"数据架构的设计与应用"论题，依次从以下三个方面进行论述：

1. 简要叙述你参与设计和开发的系统的数据架构，以及你所担任的工作；
2. 详细论述你在该系统中所采用的存储选型依据、数据模型设计方法以及一致性保障措施；
3. 分析你在数据架构设计与实施中遇到的问题及解决措施。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台为例，论述了该平台数据架构的设计与应用。该平台承载预约管理、客户消息、通知发送与主数据联动等业务，其数据形态差异极大，既有要求事务与关联查询的强一致交易数据，也有字段频繁演进的半结构化数据，还有体积较大的文件实体。我在项目中担任系统架构师，负责数据模型设计、存储选型与数据一致性方案设计。针对单一存储难以同时兼顾强一致与弹性扩展、异构存储之间的一致性难以保障、批量作业与在线业务争抢数据资源三大挑战，我采用混合存储策略，以关系型数据库承载强一

致交易，以云数据库承载高弹性半结构化数据，以对象存储承载文件实体，并通过事件驱动链路达成最终一致；同时为批量作业配置独立的读写数据源实现读写分离。系统上线后运行稳定，数据查询延迟与批量作业耗时均明显改善。

正文

一、项目背景与数据挑战

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务 H.I.S. 集团 300 余家营业门店与约 5000 名业务人员，日均处理预约约 3 万笔。我在项目中担任系统架构师，负责数据模型设计、存储选型与数据一致性方案设计。

在项目启动阶段，我首先对平台的数据进行了分类梳理，最终归纳为四类，这四类数据的特征差异构成了数据架构设计的全部依据。

第一类是强一致交易数据，包括预约、申请人、旅行者、费用明细等。这类数据之间存在复杂的关联关系，查询时往往需要多表联结，且写入必须保证事务性。第二类是字段频繁演进的半结构化数据，典型如旅行主信息——其字段随各国签证政策与集团业务规则频繁增删，若以关系表承载，每一次变更都需要执行表结构变更，代价高且风险大。第三类是文件实体，包括护照扫描件、各类文档与亲权同意书，特点是体积大、访问频率低、需要独立的对象存储能力。第四类是高读写小数据，如消息线程与流水号，读写频繁且要求低延迟。

这四类数据的存在，使“用一种存储解决所有问题”的思路从一开始就走不通：关系型数据库在强一致与关联查询上不可替代，但在字段灵活性与弹性扩展上存在明显短板；NoSQL 数据库在模式灵活与横向扩展上优势突出，却难以提供跨表事务；对象存储适合大文件，却不适合承载业务查询。此外，批量作业与在线业务访问同一数据源时，批量扫描会挤占在线业务的连接资源，这是我们在早期预演中就已发现的风险。

二、存储选型与数据模型设计

基于上述数据分类，我确立了混合存储的选型方案，选型依据是数据的一致性要求、访问模式与演进频率。

关系型数据库承载强一致交易。 预约、申请人、旅行者、费用等需要事务与复杂关联查询的数据，全部落在关系型数据库，共 79 张表，其中主数据表 26 张（m_ 前缀）、业务表 53 张（t_ 前缀）。主数据与业务数据在物理上分离组织，便于分别治理：主数据由外部主数据平台同步而来，业务侧只读；业务数据承载日常交易，读写频繁。

云数据库承载高弹性半结构化数据。 对于字段演进频繁、访问模式以单条读写为主的数据，我选择云数据库，共设计了 5 张表，分别承载旅行信息、文档元数据、亲权同意书、消息线程与流水号。

在此我重点说明**单表多实体的设计方法**。以旅行信息表为例，我采用“分区键 + 排序键 + 载荷字段”的模型：以旅行编号作为分区键，以带类型前缀的标识作为排序键（消息线程以 THR# 开头、消息以 MSG# 开头、文件以 FIL# 开头），实体内容以结构化文本形式存于载荷字段。这样设计带来三个好处：其一，一张表可以承载同一旅行下的多种数据类型，避免了表数量随业务类型线性膨胀；其二，同一旅行的全部数据落在同一分区，查询时可以通过一次分区查询取回，无需跨分区扫描，这是延迟可控的关键；其三，字段的增删只需调整载荷内容，完全不需要执行表结构变更，业务规则的快速迭代不再受阻于数据库层面。

对象存储承载文件实体。 护照、文档、亲权同意书等文件统一存放于对象存储，关系库与云数据库中只保留文件的引用信息。这一拆分使数据库的体积与备份恢复时长大幅下降。

流水号的分布式生成。 平台需要跨服务生成唯一且有序的业务编号，关系型数据库的自增机制无法满足分布式场景。为此我在云数据库中设计了一张专用的流水号表，以业务类型作为键、当前序号作为值，通过条件更新保证并发安全。

三、数据访问与一致性保障

数据访问层的组织。 关系型数据库侧统一采用 MyBatis 作为持久层框架，并通过统一版本管理插件锁定其版本，避免因版本漂移导致行为不一致。实体按主数据与业务数据分离组织，数据访问接口按业务域归类。云数据库侧通过增强客户端提供对象映射能力，通用操作封装在仓储基类中，实体属性名集中定义在常量类中，避免魔法字符串散落各处。为控制全表扫描的风险，我把扫描上限显式设置为 10000 条，为可能出现的误用设置硬边界——这个上限远高于任何正常的单次查询所需，因此它不影响正常业务，却能在代码误用导致全表扫描时及时失败而非拖垮数据库。

并发写入的控制。 云数据库侧不支持关系库那样的行锁机制，因此我采用乐观锁配合重试的策略：为需要并发控制的实体引入版本属性，更新时校验版本，冲突则以最大 4 次尝试、1 秒间隔重试。这一方案在预约信息更新等场景下运行良好。

跨存储的最终一致性。 混合存储必然带来一致性问题。写入主链路落在云数据库，而报表与既有系统仍在关系库中查询，两者需要同步。我的方案是把同步设计为事件驱动：以云数据库的数据流捕获变更事件，以消息队列承接业务消息，驱动十四个按业务域划分的函数完成同步。主链路只写云数据库即返回，同步异步进行，达成秒级的最终一致性。需要说明的是，这一设计是有意为之的权衡——预约写入等强一致场景仍走关系库事务，不做异步化；只有报表与通知这类可以容忍秒级延迟的场景才走异步链路。

主数据的本地化。 集团主数据由外部平台统一管理，若业务查询直连外部系统，则外部系统的任何抖动都会传导至业务操作。因此我设计了一个定时作业，从主数据接口拉取本国、城市、航空公司、货币等主数据，落地为本地只读副本，业务侧只查本地。数据新鲜度通过同步频率控制，这是以可用性优先的合理取舍。

四、实施中遇到的问题及解决措施

问题一：批量作业与在线业务争抢数据资源。 平台的成本文件生成、外部预约导入、待办状态更新等批量作业需要从生产库大量读取并回写结果，而在线预约接口同时也在访问同一数据库。若共享连接池，夜间批处理峰值会直接推高在线接口的延迟。我的解决方案是为批处理配置独立的读写数据源：读取使用只读账号，写入使用写账号，并通过代码包路径隔离两套数据访问接口，使读写请求在物理层面分离。同时，我把事务隔离级别设置为读已提交，减少长事务持有锁的时间；并把批量游标大小限制在 200 条，防止海量结果集一次性载入导致内存溢出——这是批处理最常见的故障模式。改造后，批量作业耗时下降约 30%，在线接口的响应时间不再受批量作业影响。

问题二：跨国部署下的时间一致性。 平台服务日本本部，源数据库运行于日本标准时间，而容器运行环境为协调世界时，开发终端又是另一个时区。三套时区并存时，预约截止时间、邮件发送时刻等跨时区比较极易出错，后果是提前发送或漏发——对旅游业务而言这是不可接受的。我的处理原则是时间口径单一化：存储层统一使用协调世界时；展示层按业务方时区转换；转换动作集中到统一的格式化工具类，禁止在业务代码中散落直接取当前时间的调用。批处理侧更为彻底——执行基准日与取得范围作为作业参数显式传入，不依赖系统时钟，使同一参数的重跑结果完全可复现。

问题三：主数据与业务数据的一致性。 初期曾出现主数据更新后业务侧仍在旧值的情况。经排查，根源是同步作业失败后没有可观测的手段，运维人员无从知晓。我为此补充了两项措施：一是把同步作业的执行结果纳入

监控，作业失败即触发告警；二是让同步作业的基准日期参数化，使补数可以精确指定日期范围重跑。改造后，主数据不一致的问题单数量下降约 90%。

五、总结与展望

平台上线至今已稳定运行约 8 个月。数据架构方面的成效主要体现在：云数据库侧的单条查询延迟稳定在 10 毫秒以内；批量作业耗时下降约 30%；数据存储成本因文件外置而下降约 25%；跨存储同步延迟稳定在秒级。

目前仍存在两点不足：一是混合存储引入了运维复杂度，多套存储的备份策略、容量规划与权限管理需要分别维护，对运维团队的要求较高；二是跨存储的一致性链路缺少统一的延迟监控，异常时的排查需要分别在多个系统中查看日志，路径偏长。后续计划引入统一的链路追踪与一致性校验作业，对关键数据定期执行对账。

通过本项目，我对数据架构设计形成了三点认识。第一，不存在"最好的存储"，只存在"与数据访问特征最匹配的存储"，选型的依据应当是数据本身的一致性要求、访问模式与演进频率，而不是技术的新旧。第二，服务边界必须与数据边界对齐——如果多个服务共享同一张表，那么无论代码如何拆分，数据层面依然是耦合的。第三，混合存储的代价必须被正视：一致性、运维复杂度与排查难度都会上升，架构师的责任是在采用之前就明确这些代价，并设计好补偿措施。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

位置	示例值	请替换为
门店数 / 业务人员数	300 余家 / 约 5000 名	真实数量
日均预约量	约 3 万笔	真实数量
关系库表数量	79 张（主数据 26 + 业务 53）	✓ 已核实
云数据库表数	5 张	✓ 已核实
单条查询延迟	10 毫秒以内	真实数据
批量游标 / 扫描上限	200 条 / 10000	✓ 已核实
批量作业耗时下降	约 30%	真实数据
存储成本下降	约 25%	真实数据
主数据问题单下降	约 90%	真实数据
稳定运行时长	约 8 个月	真实时长

本文字数

摘要约 330 字，正文约 2780 字，**合计约 3110 字。**

可复用要点

- **四类数据分类法**（强一致 / 半结构化 / 文件 / 高频小数据）是本文最有价值的方法论，可直接迁移到任何数据架构题目
- **单表多实体（分区键 + 排序键 + 载荷）** 是一个有技术深度的设计点，多数考生写不出来

- **多时区一致性** 是跨国项目的特色难点，几乎所有“数据架构”“批处理”“国际化”类题目都能用
- 若题目侧重“数据治理”，把主数据本地化与监控告警展开；若侧重“性能”，把读写分离展开

论高可用架构的设计与实现

⚠ 关于本文中的数字 本文的量化数据（可用率、响应时间、故障恢复时长等）均为**示例值**，用于展示论文应有的完整度与写法。**架构类数字（表数量、接口数、配置值）**已从项目源码核实，可直接使用；**业务规模类数字（门店数、用户数、业务量、改善幅度）**在项目资料中尚未确定，属示例值，背诵前请替换为你的**真实数据**。文末附有「待替换数字清单」，便于逐项核对。

试题

论高可用架构的设计与实现

高可用架构是指通过冗余设计、故障转移、容错处理、资源隔离等技术手段，使系统在部分组件发生故障时仍能持续对外提供服务，并将故障对业务的影响控制在可接受范围内。高可用架构设计需要明确系统的可用性目标，识别影响可用性的关键因素，并在成本与可用性之间做出合理权衡。

请围绕“高可用架构的设计与实现”论题，依次从以下三个方面进行论述：

1. 简要叙述你参与设计和开发的高可用系统，以及你所担任的工作；
2. 详细论述你在该系统中采取了哪些措施来保障系统的高可用性；
3. 分析你在可用性保障过程中遇到的问题及解决措施，并说明如何验证系统的可用性。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台为例，论述了高可用架构在该项目中的设计与实现。该平台服务日本 H.I.S. 集团营业本部与旅行者，业务具有强季节性——长假前预约量激增，平峰期迅速回落，对系统可用性提出了刚性要求。我在项目中担任系统架构师，负责可用性目标制定、故障隔离方案设计与可靠性保障机制建设。针对批量作业与在线业务争抢数据资源、消息推送存在重复发送风险、多实例部署下会话难以保持一致三大挑战，我从资源隔离、幂等重试、状态外置三个维度进行设计：为批量作业配置独立的读写数据源实现物理隔离；以执行状态的幂等校验与专用重试服务杜绝重复推送；以集中式会话存储实现 Web 层无状态化，支持任意水平扩容。系统上线后年可用率达到 99.95%，未发生因批量作业导致的在线服务降级，验证了方案的有效性。

正文

一、项目背景与可用性目标

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务 H.I.S. 集团 300 余家营业门店与约 5000 名业务人员，日均处理预约约 3 万笔、消息与通知约 12 万条。我在项目中担任系统架构师，负责可用性目标制定、故障隔离方案设计以及可靠性保障机制的建设。

旅游业务对系统可用性有着刚性要求。预约窗口期一旦系统不可用，客户会转向其他渠道，流失的订单通常无法挽回；而通知与消息若漏发或重复发送，则会直接影响客户体验与品牌信誉。因此，在项目启动阶段，我与业务方及运维方共同定义了明确的可用性目标：年可用率不低于 99.9%；全部事务的 90 分位响应时间不超过 5 秒，其中预约查询等核心场景进一步收紧至 2 秒以内；夜间批量作业必须在次日营业开始前完成；任何单一组件故障不得导致全站不可用。

在明确目标之后，我着手识别威胁可用性的关键因素。经过分析，我认为最大的威胁并非来自外部攻击或硬件故障，而是来自系统内部的三类问题：一是资源争抢，批量作业与在线业务共享数据库资源；二是重复处理，消息推送在失败重试时产生重复；三是状态耦合，多实例部署时用户的会话状态无法共享。下面分别论述我的设计。

二、高可用架构的总体设计

我的设计思路是围绕上述三类威胁逐层设防，形成资源隔离、故障域隔离、状态外置、数据层可靠性四层防护。

第一层是资源隔离。这是本项目中最关键的一层。平台的成本文件生成、外部预约导入、待办状态更新等批量作业需要从生产库大量读取并回写结果，而在线预约接口同时也在访问同一数据库。若两者共享连接池，夜间批处理峰值会直接推高在线接口的响应时间，严重时甚至导致连接耗尽。为此，我为批处理配置了独立的读写数据源：读取使用只读账号，写入使用写账号，并通过代码包路径隔离两套数据访问接口，使读写请求在物理层面彻底分离。同时，我把事务隔离级别设置为读已提交，减少长事务持有锁的时间；把批量游标大小限制在 200 条，防止海量结果集一次性载入导致内存溢出。这一层的设计牺牲了配置的简洁性，换来了在线业务的稳定，我认为这个交换完全值得。

第二层是故障域隔离。我把系统按部署单元划分为 Web、Service、Batch、FaaS 四类，使任一单元的故障不会扩散。其中批处理模块不启用 Web 容器，与在线业务没有进程级耦合；同步链路则按业务域拆分为十四个独立的函数，每个函数拥有独立镜像与独立部署单元，某个业务域的处理异常不会波及其他业务域。这一设计增加了运维单元的数量与管理成本，但把“一个模块崩溃导致全站不可用”的风险降为“单个业务域受影响”。

第三层是状态外置。Web 层多实例部署后，若会话存储在各个实例的本地内存中，负载均衡会把同一用户的请求分发到不同实例，导致用户被反复踢下线。我引入了集中式会话管理方案，把会话数据统一存放于云上无服务器形态的键值存储中，会话超时设为 28800 秒。这一取值是与业务方共同确定的：营业人员一个工作日的连续操作不应因会话过期而中断，8 小时恰好覆盖营业时段。会话外置后，Web 实例完全无状态，可以任意水平扩容与重启而不影响在线用户。

第四层是数据层可靠性。具体措施包括：对需要并发控制的实体采用乐观锁配合重试，冲突时以固定次数与固定间隔重试；把护照、文档等文件实体从数据库中剥离到对象存储，降低数据库体积与备份恢复时长；为集团主数据建立本地只读副本，使外部平台的故障不影响业务查询。

三、关键技术的实现

幂等控制，杜绝重复推送。消息与邮件推送是本平台最典型的可靠性敏感点，一旦重复发送，对客户而言就是直接的体验事故。我设计了三层防护：第一层是**执行状态幂等校验**，以专门的作业管理表记录每次作业的执行状态，同一作业处于“处理中”时重复触发会被直接拒绝——这一机制最初是为防范容器调度器的多重触发而设，后来成为全部作业的统一防线；第二层是把重试能力独立为专用的重试服务，与业务逻辑彻底解耦，最大尝试 4 次、间隔 1 秒，使重试策略可以独立调整而无需改动业务流程；第三层是投递状态跟踪表，记录每一次投递的结果（未投递 / 已投递），使异常场景下能够精确定位漏发与重发的范围。此外，我把事务边界拆分为独立的服务类，把事务粒度控制在必要范围内，避免大事务回滚造成连带重放。

可观测性建设。高可用的前提是“故障能被及时发现”。批处理属于短生命周期的进程，作业结束后进程即退出，常规的进程监控无法采集其运行状态。为此我引入了指标推送机制，由作业主动把执行指标推送到 Prometheus 的 PushGateway，并把关闭动作设为“进程退出前投递”，确保最后一批指标不会随进程退出而丢失。函数侧则借助运行时框架输出结构化日志与链路追踪信息。这样一来，无论是批量作业失败还是同步链路异常，运维人员都能在监控面板上第一时间发现。

作业的可重跑与可复现。批处理的重跑能力是可靠性的重要保障。我把执行基准日、取得范围、导入类型等关键参数定义为作业参数显式传入，而不依赖系统时钟取当前日期。这一设计带来两个收益：一是补数时可以指定历史日期重跑，结果与当日运行完全一致；二是测试时可以固定参数，使结果可预期。对于跨国部署而言，这一点尤为重要——服务日本本部而容器运行于协调世界时，若依赖系统时钟判断“今天”，跨时区下极易取到错误的日期边界。

四、实施中遇到的问题及解决措施

问题一：批量作业峰值冲击在线服务。系统上线前的压测中，我们发现夜间批量作业执行期间，在线预约查询接口的响应时间由 300 毫秒劣化至 2.5 秒，已超出可用性目标。经分析，根源是两者共享数据库连接池，批量扫描占用了大量连接。我在上文的资源隔离方案正是针对此问题设计的——通过对批处理配置独立的读写数据源并按包路径隔离数据访问接口，使两者在物理层面分离。改造后再次压测，批量作业执行期间在线接口的响应时间稳定在 320 毫秒以内，批量作业自身耗时反而下降约 30%，实现了双赢。

问题二：消息重复推送。上线初期，我们收到客户反馈称同一封通知邮件收到了两次。排查发现，是邮件网关超时后系统自动重试，而首次请求实际上已经成功，重试造成了重复发送。这一事件的本质是分布式环境下的幂等性问题，而非简单的异常处理问题。我的解决方案即上文第三节所述的执行状态幂等校验、专用重试服务与投递状态跟踪三层防护。改造后，重复发送率由约 0.3% 降至 0.01% 以下。

问题三：多实例部署下的会话丢失。平台上线后随业务量增长需要横向扩容 Web 实例，但扩容后用户频繁被强制重新登录。原因是会话数据存储在单个实例的本地内存中，请求被负载均衡分发到其他实例时无法识别会话。我通过引入集中式会话存储解决该问题，并额外获得了一项安全收益：会话数据集中在服务端，可以强制失效任意用户的会话，从而实现“管理员踢出用户”“用户改密后全端下线”这类安全能力。

关于可用性的验证方式。我们采取了三层验证：一是在开发阶段利用容器化技术与本地云模拟工具搭建与云端一致的依赖环境，对读写分离、幂等控制等关键机制编写集成测试；二是在上线前进行压力测试，重点验证批量作业与在线业务并发执行时的相互影响；三是在运行期通过监控指标持续观测，把可用性目标拆解为可量化的监控项并设置告警阈值。

五、总结与展望

平台上线至今已稳定运行约 8 个月，年可用率达到 99.95%，超过了既定目标；批量作业均在次日营业开始前完成，未发生因批量作业导致的在线服务降级；消息与通知的重复发送率控制在 0.01% 以下。

目前仍存在两点不足：一是跨存储的最终一致性链路尚无统一的延迟监控，异常时的排查路径偏长；二是无服务器计算在低频函数上存在冷启动延迟，尚未引入预置并发配置。此外，需要客观说明的是，本平台目前采用的可靠性手段以资源隔离、故障域隔离、状态外置与幂等重试为主，尚未引入熔断与限流机制。在流量突增时，系统缺乏主动的自我保护能力，这是我们后续需要补齐的能力。

后续计划引入一致性延迟监控、预置并发配置以及服务端的熔断限流能力，并定期开展故障演练，验证在真实故障场景下的恢复能力。

通过本项目，我认识到高可用并不是靠某一项技术实现的，而是由一系列设计共同构成的体系：用资源隔离消除内部争抢，用故障域隔离限制故障扩散，用状态外置支撑水平扩容，用幂等与重试应对不可避免的失败。更重要的是，高可用必须有明确的目标与验证手段——没有量化目标的“高可用”只是一句口号，而无法验证的高可用则无法证明其存在。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

位置	示例值	请替换为
门店数 / 业务人员数	300 余家 / 约 5000 名	真实数量
日均预约量 / 消息量	约 3 万笔 / 约 12 万条	真实数量
年可用率目标 / 实际	99.9% / 99.95%	真实指标
响应时间目标	90 分位 5 秒以内（核心场景 2 秒）	✓ 已核实
压测劣化情况	300ms → 2.5s	真实压测数据
改造后响应时间	320 毫秒以内	真实压测数据
批量作业耗时下降	约 30%	真实数据
重复发送率	0.3% → 0.01%	真实数据
会话超时	28800 秒（8 小时）	✓ 已核实
重试策略	最大 4 次 / 间隔 1 秒	✓ 已核实
批量游标	200 条	✓ 已核实

本文字数

摘要约 330 字，正文约 2820 字，**合计约 3150 字**。

可复用要点

- **开头必写「可用性目标」**——这是高可用类论文的标准起手式，没有目标的论文会被判定为空谈
- **「高可用最大的威胁来自内部资源争抢而非外部攻击」** 是一个有洞察力的判断，能显著提升论文层次
- **结尾主动写出「尚未引入熔断限流」** 这一不足，既真实又体现架构判断力，比硬编造用了熔断组件安全得多
- 若题目问“如何验证可用性”，直接引用第四节末段的三层验证方式

论信息系统安全架构的设计

⚠ 关于本文中的数字 本文的量化数据（用户规模、权限变更耗时、安全事件等）均为**示例值**，用于展示论文应有的完整度与写法。**架构类数字（表数量、接口数、配置值）**已从项目源码核实，可直接使用；**业务规模类数字（门店数、用户数、业务量、改善幅度）**在项目资料中尚未确定，属示例值，背诵前请替换为你的**真实数据**。文末附有「待替换数字清单」，便于逐项核对。

试题

论信息系统安全架构的设计

信息安全架构是指为保障信息系统的机密性、完整性和可用性，在身份认证、访问控制、数据保护、安全审计等方面所进行的整体性设计。随着信息系统承载的数据日益敏感、与外部系统的交互日益频繁，安全架构设计已成为系统架构设计中不可分割的组成部分。安全架构设计应当遵循最小权限、纵深防御、默认拒绝等基本原则。

请围绕“信息系统安全架构的设计”论题，依次从以下三个方面进行论述：

1. 简要叙述你参与设计和开发的信息系统，以及你所担任的工作；
2. 详细论述你在该系统中如何从身份认证、访问控制、数据保护与安全审计等方面进行安全架构设计；
3. 分析你在安全设计与实施中遇到的问题及解决措施。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台为例，论述了该平台安全架构的设计与应用。该平台承载旅行者护照信息、亲权同意书、紧急联系人等敏感个人数据，并需与集团主数据平台、外部预约系统、邮件与消息网关等多个外部系统交互，安全要求高、合规约束严。我在项目中担任系统架构师，负责身份认证体系设计、权限模型设计与凭证管理方案制定。针对多角色细粒度权限难以管控、跨系统凭证存在泄露风险、敏感数据在传输与存储中需全面防护三大挑战，我从统一身份认证、默认拒绝的授权设计、临时凭证与加密传输三个维度进行设计：基于 OAuth2 与 OIDC 协议实现统一登录，账号生命周期由集团统一治理；把授权校验由逐个放行改为默认拒绝并下沉至接口层；以临时凭证替代长期访问密钥，敏感凭据统一收口。系统上线后未发生安全事件，权限调整效率显著提升。

正文

一、项目背景与安全需求

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务 H.I.S. 集团 300 余家营业门店与约 5000 名业务人员。我在项目中担任系统架构师，负责身份认证体系设计、权限模型设计以及凭证与密钥的管理方案。

安全架构的设计必须从“保护什么”出发，否则后续的一切措施都失去依据。因此我首先对平台承载的数据进行了分级，将其归纳为三类。第一类是个人身份信息，包括旅行者的护照信息、姓名、出生日期与联系方式。第二类是特殊敏感数据，包括未成年人的亲权同意书以及旅行者的紧急联系人信息——这类数据的敏感性高于一般个人信息，一旦泄露可能危及人身安全。第三类是业务敏感数据，包括预约成本、客户消息内容等，泄露会直接影响商业利益。

除数据本身外，平台还面临复杂的交互面：向上需要与集团主数据平台同步数据，向外需要对接外部预约系统，向下需要调用邮件网关与消息推送服务。每一个交互面都是一个潜在的入口。合规方面，平台需要遵循日本个人信息保护法的相关要求。

基于以上分析，我识别出三个核心挑战：一是平台角色众多、业务域差异大，细粒度的权限控制难以管控；二是平台需要访问云上的多项服务与外部系统，凭证一旦长期固定存放，泄露风险将随时间累积；三是敏感数据在传输与存储的各个环节都需要防护，任何一处疏漏都可能导致前功尽弃。下面分别论述我的设计。

二、安全架构的总体设计

我按照身份认证、访问控制、数据安全、安全审计四个层次组织安全架构，遵循纵深防御与默认拒绝的基本原则。

第一层是身份认证。平台采用 OAuth2 与 OIDC 协议实现统一身份认证，Web 层通过标准的 OAuth2 客户端接入集团统一认证服务，以授权码模式完成登录，令牌的校验基于 JOSE 标准实现。用户身份属性直接从令牌中提取，平台不维护本地用户表——这样设计的目的是让账号的创建、变更与注销全部由集团统一治理，避免出现“人已离职但系统账号仍在”这类常见的管理漏洞。同时，我通过视图层的安全扩展把安全上下文暴露到页面渲染中，使页面元素的显隐可以直接由权限决定，而不是依赖前端脚本隐藏——后者只是“看不见”，前者才能真正做到“点不到”。

需要说明的是，引入外部认证服务也带来了可用性上的依赖风险。我的应对措施是把用户信息在登录后缓存于服务端会话中，使认证服务短暂不可用时，已登录用户仍可继续操作，避免认证服务成为系统的可用性单点。

第二层是访问控制。平台的授权模型采用基于角色的访问控制与接口级权限标识相结合的方式。角色维度划分为系统管理员、业务操作员、主管、贵宾客户等，各角色持有不同的权限集合。关键在于校验位置的选择：我把权限校验统一下沉到接口层，由安全框架对所有对外接口统一拦截，而不是在各控制器方法上逐个添加注解。

第三层是数据安全。传输方面，平台对外的 Web 访问全站启用加密传输，内部服务之间的远程调用同样使用加密通道。在入口侧，平台还通过基础设施即代码的方式定义了 Web 应用防火墙规则，对常见的 Web 攻击特征进行拦截，构成应用之外的第一道边界防护——这一层的价值在于，它不依赖应用代码的正确性，即使应用存在未知缺陷也能提供缓冲。凭证方面，平台不使用长期固定的云服务访问密钥，而是通过安全令牌服务获取临时凭证，配合默认凭证链兜底，使凭证自动轮换且有效期极短。配置方面，函数侧的数据库连接、外部端点等全部从云参数服务读取，代码仓库中不保留任何明文凭据；服务端的凭证获取也统一收口到配置类中。存储方面，文件实体统一存放于对象存储，数据库仅保存文件引用；数据库账号按读写职责分离，批量作业使用的读账号不具备写权限。

第四层是安全审计。平台在 Web 层设置了独立的审计能力，关键业务操作——预约变更、消息发送、主数据修改——均记录操作者、操作时间与操作内容。审计日志与业务数据分离存储，避免业务数据回滚时审计记录同时丢失。

三、关键技术的实现

会话安全。平台的会话由集中式会话管理方案统一维护，存储于云上无服务器形态的键值存储中，超时时间为 28800 秒——这一取值是与业务方共同确定的：营业人员一个工作日的连续操作不应因会话过期而中断。相比本地会话，集中式会话带来两点安全收益：一是会话数据不落在应用服务器的磁盘上，降低了主机被入侵后会话被窃取的风险；二是可以在服务端强制失效任意会话，从而实现管理员强制下线、用户改密后全端下线等安全能力。

凭证与密钥的集中治理。我把全部凭证获取动作统一收口到少数几个配置类中，业务代码不直接接触凭证。这一做法看似简单，价值却很大：凭证轮换、加密算法升级、权限范围调整都只需在单一位置修改，不会遗漏调用点。与之配套的是环境隔离机制——云数据库的表名统一携带环境前缀，实现“一个账户一个集群”的隔离，从根本上防止测试环境误连生产数据。

接口契约中的安全边界。由于平台通过统一的接口契约向 Java、Go、PHP 三端提供服务，契约本身也承担着安全职责。我为每个远程调用接口声明了统一的 HTTP 映射路径，使其便于在接入层集中实施鉴权与限流；同时定义了统一的错误返回结构，对外只暴露错误码与友好提示，不泄漏堆栈信息与内部实现细节。平台的错误码体系

以消息编号的形式统一定义，错误信息由配置文件自动生成常量类，既避免了硬编码，也保证了不同语言调用方对错误含义的一致理解。

输入校验与内容净化。平台在 Web 层启用了参数校验框架，对入参进行类型与范围校验；数据库访问全部通过参数绑定实现，从根本上杜绝了字符串拼接导致的注入风险。在涉及富文本内容处理的场景，则使用专门的 HTML 净化工具对内容进行处理，防止存储型跨站脚本攻击——这一点在客户消息场景尤为重要，因为消息内容会在客户页面中回显。

四、实施中遇到的问题及解决措施

问题一：新增接口的权限遗漏风险。项目初期，权限校验采用在控制器方法上逐个添加注解的方式。随着接口数量增长——平台 Web 层共有 64 个控制器类，分布在 18 个代码包下——这种方式暴露出明显缺陷：新增接口时若忘记添加权限注解，该接口将处于无保护状态，而这类漏洞在功能测试中不会显现，只有在被利用时才会暴露。我的判断是，这不是开发人员的疏忽问题，而是**授权策略的默认值设计错误**。因此我把策略从“逐个放行”改为“默认拒绝”：在安全框架中统一拦截全部对外接口，未显式声明权限的路径一律拒绝；权限以标识集合的形式随令牌下发，接口层按标识校验，而非按角色硬编码判断。改造后，新增接口默认处于受保护状态，权限遗漏的风险被从根本上消除；同时，角色的调整从“修改代码”变为“调整配置”，权限变更的耗时由平均 2 天缩短至半天以内。

问题二：长期密钥的泄露风险。项目初期，平台访问云服务使用的是长期固定的访问密钥，存放于配置文件中。随着接入的云服务增多，密钥数量不断增加，轮换成本高，且一旦代码仓库或配置文件泄露，风险将长期存在。我推动了两项改造：一是把云服务的访问方式改为通过安全令牌服务获取临时凭证，凭证自动轮换且有效期极短，即使泄露其危害窗口也极为有限；二是把函数侧的全部凭据迁移到云参数服务，代码仓库中不再保留任何明文凭据。改造后，凭证管理的动作从“定期人工轮换”变为“系统自动完成”。

问题三：认证服务的可用性依赖。引入集团统一认证服务后，我们担心认证服务故障会导致平台无法登录。经过分析，我认为完全消除这一依赖并不现实，但可以把影响范围控制住：对于已登录用户，其身份信息缓存在服务端会话中，认证服务短暂不可用不影响其继续操作；只有新登录请求会受影响。这样把“全站不可用”降级为“新用户暂时无法登录”，影响面大幅缩小。

五、总结与展望

平台上线至今已稳定运行约 8 个月，服务约 5000 名业务人员，未发生安全事件。安全架构方面的成效主要体现在：授权策略改为默认拒绝后，新增接口的权限遗漏风险被消除；凭证改为临时获取后，人工轮换工作完全取消；权限调整由改代码变为改配置，平均耗时缩短约 75%。

目前仍存在两点不足：一是权限模型目前以角色的权限标识组合为主，尚未实现数据级（行级）的细粒度控制，即无法限制某个操作员只能查看自己负责的客户数据；二是跨语言接口契约的破坏性变更尚未引入自动化的安全影响评估，仍依赖人工评审。后续计划引入数据级授权能力与契约变更的自动化审计。

通过本项目，我对安全架构形成了三点认识。第一，安全架构的核心不是堆砌安全产品，而是建立“默认拒绝”的基线——安全的默认值应当是拒绝，而不是放行。第二，凭证、权限与审计这三样东西必须收口到可治理的位置，散落各处的安全措施等同于没有措施。第三，安全与可用性之间常常存在张力，架构师的责任不是追求某一方面的极致，而是明确每一处取舍的影响范围，并把它控制在可接受的边界内。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

位置	示例值	请替换为
门店数 / 业务人员数	300 余家 / 约 5000 名	真实数量
控制器类 / 代码包数量	64 个 / 18 个	✓ 已核实
权限变更耗时	平均 2 天 → 半天以内	真实数据
权限变更效率提升	约 75%	真实数据
会话超时	28800 秒 (8 小时)	✓ 已核实
稳定运行时长	约 8 个月	真实时长
安全事件	未发生	如实填写

本文字数

摘要约 340 字，正文约 2800 字，合计约 3140 字。

可复用要点

- 「保护什么 → 威胁是什么 → 措施是什么」 是安全类论文的唯一正确组织方式，切忌罗列安全产品
- 数据三级分类（个人身份信息 / 特殊敏感数据 / 业务敏感数据）是本文的亮点，体现合规意识
- 「默认拒绝」的重构 是最有分量的架构决策，几乎所有安全类题目都可作为核心难点
- 「认证服务降级为只影响新登录」 体现了安全与可用性的权衡思维
- Web 应用防火墙 在本项目中真实存在（由基础设施即代码定义），文中已按"不依赖应用代码正确性"的角度表述，可放心使用
- ⚠ 本文刻意未编造堡垒机、入侵检测、国密算法、等级保护认证等本项目并未使用的内容。若你的项目确实有，可自行补充；若没有，保持现状比编造更安全

论系统集成方法及其应用

⚠ 关于本文中的数字 本文的量化数据（接口数量、联调返工率、集成周期等）均为示例值，用于展示论文应有的完整度与写法。架构类数字（表数量、接口数、配置值）已从项目源码核实，可直接使用；业务规模类数字（门店数、用户数、业务量、改善幅度）在项目资料中尚未确定，属示例值，背诵前请替换为你的真实数据。文末附有「待替换数字清单」，便于逐项核对。

试题

论系统集成方法及其应用

系统集成是指将不同的软件系统、硬件设备、网络通信等要素组合成一个有机整体，使其能够协同工作、信息共享的过程。在企业信息化建设中，系统集成是打通信息孤岛、实现业务贯通的关键环节。系统集成需要解决接口

标准不统一、数据格式不一致、通信协议异构、集成双方技术栈差异等一系列技术问题，同时还需要考虑集成的可维护性与可扩展性。

请围绕"系统集成方法及其应用"论题，依次从以下三个方面进行论述：

1. 简要叙述你参与设计和开发的系统集成项目，以及你所担任的工作；
2. 详细论述你在该系统中采用的系统集成方法，以及如何解决集成中的关键技术问题；
3. 分析你在系统集成过程中遇到的问题及解决措施。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台为例，论述了系统集成方法在该项目中的应用。该平台需要与集团主数据平台、外部预约系统、邮件网关、消息推送服务等多个异构系统集成，同时自身能力须以统一接口向 Java 内部服务、Go 周边平台与 PHP 既有系统三类调用方开放，集成面广、参与方技术栈差异大。我在项目中担任系统架构师，负责集成方案设计、接口契约定义与集成治理机制建设。针对集成各方接口定义分散导致口径漂移、跨语言产物无法保证一致、同步接口难以覆盖异步通知场景三大挑战，我采用"契约先行、单一来源、多端生成"的集成方法：以 Protocol Buffers 作为唯一契约源，按服务定义、请求、响应、枚举与可复用消息五层组织，由同一份契约驱动三语言客户端与在线接口文档的生成，使集成各方的接口定义在构建期即被强制对齐。集成完成后跨端联调返工减少约 80%，接口变更的沟通成本显著下降。

正文

一、项目背景与集成需求

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务 H.I.S. 集团 300 余家营业门店与约 5000 名业务人员，日均处理预约约 3 万笔。我在项目中担任系统架构师，负责集成方案设计、接口契约定义与集成治理机制的建设。

平台并非一个孤立系统，而是一个集成中枢，其集成需求来自三个方向。

第一个方向是向上集成集团主数据平台。 国家、城市、航空公司、货币等主数据由集团统一管理，平台需要定期获取并保持同步。集成方式为定时批量拉取，对实时性要求不高，但对完整性与可重跑性要求高。

第二个方向是横向集成外部业务系统。 平台需要对接外部预约系统导入预约数据，需要调用邮件网关发送通知邮件，需要调用消息推送服务向客户发送消息。这些系统的技术栈、协议与可用性水平各不相同，是我们无法控制的外部依赖。

第三个方向是向下开放自身能力。 这是本项目集成工作中最具挑战性的部分。平台的核心能力需要同时向三类调用方开放：平台自身的 Java 服务内部调用、集团 Go 语言构建的周边平台、以及 PHP 编写的既有业务系统。三者的技术栈、序列化方式与构建工具链完全不同。

综合来看，平台对外提供 17 个远程调用接口，集成参与方涉及 3 种语言、4 类外部系统。集成工作的成败，直接决定了平台能否真正融入集团的业务生态。

二、系统集成方案的总体设计

在方案设计阶段，我首先排除了两种常见但不可取的做法。

第一种做法是"各自维护接口定义"——即各调用方根据文档自行编写接口代码。这种做法的问题在于，接口定义一旦存在多个副本，就必然随时间漂移，而漂移在编译期不可见，只有到联调或运行时才暴露。

第二种做法是"以某一端的实现为准，其余端适配"——看似省事，实则把某一技术栈的细节变成了全局标准，长期看会造成其他端的适配负担累积。

因此我确立了"契约先行、单一来源、多端生成"的集成方法，其核心思想是：**接口契约独立于任何一端的实现而存在，所有集成参与方的接口代码均由同一份契约生成，而不是由各方自行编写。**具体分为三个层次。

第一层是契约的组织方式。我把平台的全部服务接口定义在独立的规格模块中，与任何业务实现模块都不产生依赖。契约以 89 个 proto 文件按职责分为五层组织：服务定义层（1 个）声明远程调用的服务与方法；请求消息层（17 个）定义入参结构；响应消息层（17 个）定义返回结构；枚举层（12 个）定义业务枚举值；可复用消息层（42 个）沉淀被多个接口引用的公共结构，如错误信息、地址、旅行者、护照等。分层的目的在于控制重复——在没有复用层的情况下，同一个地址结构可能在十几个接口中被重复定义，任何一处修改都会引发大面积的不一致。

第二层是产物的生成链路。契约只定义一次，但需要生成多种产物。我通过构建脚本编排了一条完整的生成链路：首先由 protobuf 插件生成 Java 产物并发布至内部 Maven 仓库，供平台内部服务使用；其次在容器化的 protoc 环境中生成 Go 产物，发布为 Go 模块，供集团周边平台使用；随后生成 PHP 产物并发布为 Composer 包，供既有业务系统使用；最后由 protoc 生成 OpenAPI 描述文件，再由文档生成工具构建为在线接口文档站点，供外部集成方与业务方查阅。四类产物全部源自同一份契约，且 Go 与 PHP 产物均在容器化的 protoc 环境中生成，以保证不同开发机器上的产出完全一致。

第三层是通信协议的选择。对于内部服务之间的调用，我选择了 gRPC 而非 REST。理由有三：其一，gRPC 基于 HTTP/2 多路复用，在高频内部调用场景下连接利用率更高；其二，采用二进制编码，序列化开销与报文体积更小，这对预约高峰期的性能表现有实际意义；其三，也是最重要的一点，protobuf 天然提供强类型契约，这与"契约先行"的方法论高度契合——如果用 REST 加 JSON，契约就只能靠文档约束，而不是靠工具链强制。

对于面向浏览器与外部集成方的场景，则由 Web 层暴露 REST 接口，接口文档由契约自动生成，兼顾了浏览器的友好性与文档的准确性。

三、关键技术问题的解决

问题一：接口命名与路径口径的统一。在集成实践中，一个容易被忽视的问题是接口的访问路径。若各服务自行定义路径规则，接入层的鉴权、限流与路由策略就难以统一实施。我在契约中为每个远程调用接口显式声明了统一的 HTTP 映射路径，全部采用"以方法名构成路径、请求体整体绑定"的同一形式，使全部接口在接入层呈现出可预测的路径规范。这一约定带来的直接收益是：鉴权、限流、日志这些横切关注点可以在接入层一次性配置，而不需要在每个服务中重复实现。

问题二：错误语义的跨语言对齐。集成中最棘手的问题之一不是"成功路径"，而是"失败路径"。若各端对错误的表达方式不一致，调用方就无法编写可靠的错误处理逻辑。为此，我推动建立了统一的错误码体系：平台的错误码以消息编号的形式统一定义（API 侧共 77 个），错误信息由配置文件自动生成常量类，各端共享同一套编号语义。同时，我定义了统一的错误返回结构，对外只暴露错误码与面向用户的友好提示，不泄漏堆栈信息与内部实现细节。这样一来，Java、Go、PHP 三端对同一个错误的判定逻辑完全一致，且都获得了"信息不泄漏"的安全保障。

问题三：异步集成场景的覆盖。单纯的同步接口无法覆盖全部集成场景。例如通知发送，若采用同步调用，调用方必须等待邮件网关返回，既拉长了响应时间，也把外部系统的故障传导给了调用方。因此我采用了同步与异步

并存的集成方式：需要即时结果的操作走 gRPC 同步调用；通知、消息这类“提交即可”的操作则通过消息队列异步投递，由平台内部的服务消费后完成发送。异步集成引入了新的问题——重复投递。对此我设计了三层防护：以执行状态幂等校验拒绝重复触发，以独立的重试服务统一重试策略（最大尝试 4 次、间隔 1 秒），以投递状态跟踪表记录每一封邮件的投递结果、重跑时只处理未投递的记录。

问题四：服务端的统一分发设计。 集成接口数量达到 17 个之后，若在每个接口上重复编写请求解析、日志记录、异常转换等样板代码，不仅冗余，而且极易在新增接口时遗漏某个横切逻辑。我在服务端采用单一入口类承接全部远程调用，通过统一的执行方法将请求分发到各业务实现类。这样，日志记录、异常转换、调用上下文校验都在分发层统一处理，各业务实现类只需关注业务逻辑本身。同时，我把被多个业务域共用的能力——文档、消息线程、亲权同意书、序列号、旅行信息——下沉为共享服务，避免同一逻辑在多个接口中重复实现。

四、集成过程中遇到的问题及解决措施

问题一：跨语言接口契约的一致性维护。 项目初期，各端曾各自根据文档维护接口定义，导致同一字段在不同端的命名与类型出现差异，直到联调阶段才被发现。我的判断是，这不是开发人员疏忽，而是**契约缺乏单一来源**这一架构缺陷。因此我没有停留在逐处修正，而是把接口定义全部收敛到规格模块，使接口变更必须通过该模块提交；同时把三语言产物的生成纳入构建流水线，让接口不一致问题从“运行时联调发现”提前到“构建期暴露”。改造后，接口层面的联调返工减少约 80%。

问题二：依赖版本漂移引发的集成失败。 平台初期各模块独立声明依赖版本，部分组件采用了动态版本区间。一次依赖升级后，protobuf 官方公共类型库被解析到一个候选版本，导致 gRPC 服务启动直接失败。这次故障让我意识到，我们虽然统一了业务接口的契约，却忽略了**第三方依赖版本这一隐式契约**——集成双方即使业务接口完全一致，也可能因为底层库版本不同而无法互通。因此我建立了独立的统一版本管理插件，各模块通过插件接入统一的版本源，并对关键组件使用强制版本策略加以锁定。改造后，同一提交在任何机器上的构建产物完全一致。

问题三：外部系统的可用性拖累。 邮件网关与消息推送服务是外部依赖，其抖动会直接影响平台的通知能力。初期我们采用同步调用，外部超时会阻塞平台线程。改造为异步投递后，外部系统的响应时间不再影响平台的主链路，但引入了重试与重复问题——这一权衡是有意义的：以最终一致性换取主链路的稳定性，同时用幂等机制控制重复的副作用。

问题四：集成变更的沟通成本。 集成参与方分属不同团队，一次接口变更往往需要多方协调。改造后，由于契约变更必须通过规格模块提交，变更的影响范围可以在构建阶段被自动识别——哪个语言的产物需要重新发布、哪些调用方会受影响，都可以从构建结果中直接读出，而不必依赖人工梳理。这使集成变更的沟通从“逐个确认”变为“以构建结果为准”。

五、总结与展望

平台集成工作完成至今已稳定运行约 8 个月。集成方面的成效主要体现在：17 个接口在三种语言之间保持口径一致，跨端联调返工减少约 80%；接口文档由契约自动生成，文档与实现不同步的问题被消除；统一版本管理插件的引入使构建具备了可复现性，因依赖版本漂移导致的集成失败不再发生。

目前仍存在两点不足：一是虽然契约有了统一来源，但破坏性变更尚未实现自动化的兼容性检查，仍需人工评审；二是异步集成链路缺少统一的端到端追踪，一次跨系统的消息流向排查需要串联多方日志。后续计划引入契约兼容性校验工具，并完善异步链路的追踪能力。

通过本项目，我对系统集成形成了三点认识。第一，系统集成的本质不是“连通”，而是“约定”——把接口的定义权从各参与方收归到一个共同认可的位置，是集成能否长期稳定的分水岭。第二，契约不仅要覆盖业务字段，还

要覆盖命名规范、错误语义与版本策略这些看似边缘的内容，因为集成故障往往就发生在这些地方。第三，同步与异步的选择应当由业务语义决定，而不是由技术偏好决定：需要即时结果的就同步，只需最终完成的就异步，而异步带来的副作用必须用幂等机制加以控制。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

已从源码核实，可直接使用

位置	真实值
契约文件数 / 分层	89 个 proto / 五层
远程调用接口数量	17 个
集成语言种类	Java / Go / PHP 三种
错误码数量	API 侧 77 个（以消息编号统一定义）
重试策略	最大尝试 4 次、间隔 1 秒

项目中不存在，需你替换

位置	示例值	请替换为
门店数 / 业务人员数	300 余家 / 约 5000 名	真实数量
日均预约量	约 3 万笔	真实数量
联调返工减少	约 80%	真实数据
重复发送率	0.01% 以下	真实数据
稳定运行时长	约 8 个月	真实时长

本文字数

摘要约 340 字，正文约 2830 字，合计约 3170 字。

可复用要点

- 「**契约先行、单一来源、多端生成**」是本文的核心方法论，一句话即可概括整篇的立场
- 「**集成故障往往不在业务字段，而在命名规范、错误语义与版本策略**」是有洞察力的判断，能显著提升论文层次
- 「**第三方依赖版本也是一种隐式契约**」这个视角非常独特，几乎所有集成类题目都能作为亮点使用
- 本文素材最扎实（proto 三语言契约是项目真实架构），**若只准备一篇，优先精背本篇**
- 若题目问“集成方法有哪些”，可补充说明数据集成、界面集成、应用集成、业务流程集成四种层次，再落到本项目

论软件架构评估方法及其应用

⚠ 关于本文中的数字 本文的量化数据（评估轮次、改进项数量、效果提升等）均为**示例值**，用于展示论文应有的完整度与写法。**架构类数字（表数量、接口数、配置值）**已从项目源码核实，可直接使用；**业务规模类数字（门店数、用户数、业务量、改善幅度）**在项目资料中尚未确定，属示例值，背诵前请替换为你的**真实数据**。文末附有「待替换数字清单」，便于逐项核对。

试题

论软件架构评估方法及其应用

软件架构评估是指在架构设计完成后或架构演化过程中，对架构方案是否满足系统的质量属性需求进行系统化评价的过程。架构评估能够及早发现架构设计中的风险与缺陷，避免在系统实现甚至上线之后才发现问题，从而大幅降低返工成本。常用的架构评估方法包括架构权衡分析法（ATAM）、软件架构分析法（SAAM）以及成本效益分析法（CBAM）等。

请围绕“软件架构评估方法及其应用”论题，依次从以下三个方面进行论述：

1. 简要叙述你参与设计和开发的系统，以及你所担任的工作；
2. 详细论述你在该系统中采用的架构评估方法，包括评估的步骤与质量属性场景的构建方法；
3. 分析架构评估的结论，以及你是如何根据评估结果改进架构设计的。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台为例，论述了架构权衡分析法（ATAM）在该项目中的应用。该平台服务日本 H.I.S. 集团营业本部与旅行者，承载预约管理、通知发送、消息推送与主数据联动等核心业务，对性能、可用性、安全性与可修改性均有明确的质量属性要求。我在项目中担任系统架构师，负责质量属性需求的梳理、架构评估的组织与评估结论的落地。在架构设计完成后，我组织业务方、开发团队与运维团队共同实施了一轮架构评估：先以效用树的形式逐层分解质量属性并标注优先级，再以“刺激源—刺激—环境—制品—响应—响应度量”六要素构建具体场景，最后通过场景走查识别出 12 个敏感点、4 个权衡点与 3 个风险点。针对评估发现的读写资源争抢、依赖版本漂移、认证服务单点等问题，我分别采取了读写数据源隔离、统一版本管理插件、用户信息会话缓存三项改进措施，使关键接口响应时间与构建可复现性均得到明显改善。

正文

一、项目背景与评估动因

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务 H.I.S. 集团 300 余家营业门店与约 5000 名业务人员，日均处理预约约 3 万笔、消息与通知约 12 万条。我在项目中担任系统架构师，负责质量属性需求的梳理、架构评估的组织以及评估结论的落地。

在架构方案初步成型后，我决定在投入大规模编码之前组织一轮正式的架构评估，动因有三个。

其一是**质量属性之间存在真实的冲突**。业务方希望预约查询尽可能快，运维方希望批量作业尽可能稳，安全方希望每一次访问都做完整校验。这些诉求单独看都合理，但合在一起就会互相牵制。仅靠架构师的经验取舍，很难让各方信服，也难以保证没有遗漏。

其二是**架构决策的影响往往在编码后期才显现**。例如我们最初计划让批量作业与在线业务共用数据库连接池，这个决策在编码阶段看不出问题，只有在夜间批处理峰值与在线请求并发时才暴露。评估的价值正在于把这类问题提前到设计阶段发现。

其三是**参与方众多，需要建立共识**。平台涉及业务、开发、运维、安全与集团多个团队，各方的关注点不同。评估过程本身就是一个建立共同理解的过程。

在多种评估方法中，我选择了 ATAM。理由是：SAAM 侧重于可修改性，对性能、可用性等属性覆盖不足；CBAM 侧重于经济决策，适用于方案选型阶段而非架构评价；而 ATAM 能够同时并在多个质量属性之间进行权衡分析，并且明确产出敏感点、权衡点与风险点，正好对应我们“多属性冲突”的核心诉求。

二、评估方法的设计与实施步骤

我组织的评估共分五个阶段，参与者包括业务方代表、开发骨干、运维负责人与我本人。

第一阶段：介绍评估方法与业务驱动因素。我首先向参与方说明 ATAM 的基本概念与分析流程，使各方建立共同语言；随后由业务方阐述平台的业务目标、约束条件与主要利益相关方。这一阶段产出的关键结论有三条：旅游业具有强季节性，预约高峰期的性能是业务的刚性要求；平台的可用性直接影响订单转化，不可用即意味着直接损失；平台需长期承载业务规则变化，可修改性关乎长期成本。

第二阶段：介绍架构方案。我以视图的形式向参与方介绍了架构方案，包括部署视图（Web、Service、Batch、FaaS 四类部署单元）、数据视图（关系库承载强一致交易、云数据库承载半结构化数据、对象存储承载文件实体）、通信视图（内部 gRPC、对外 REST）以及集成视图（契约单一来源，三语言产物生成）。

第三阶段：构建质量属性效用树。这是 ATAM 的核心环节，也是我投入精力最多的部分。效用树的构建方法是自上而下逐层细化：第一层是“效用”，第二层是质量属性类别，第三层是属性的细化方向，第四层是具体的质量属性场景，并为每个场景标注优先级（高、中、低）与实现难度（高、中、低）。

我们最终形成的效用树包含四个属性分支。性能分支下细化了两个方向：预约查询的响应时间、批量作业的执行时长。可用性分支下细化了三个方向：批量作业不影响在线服务、消息不重复不遗漏、单实例故障不影响全站。安全性分支下细化了三个方向：越权访问的抵御、凭证泄露的抵御、敏感数据传输的防窃听。可修改性分支下细化了三个方向：业务规则变化的支持、多语言调用方的接口变更支持、新门店上线的支持。

第四阶段：场景走查与架构分析。我逐个场景地走查架构方案，判断该场景能否被当前设计满足，并据此识别四类要素。

第五阶段：结果汇总与优先级排序。我把识别出的要素按处理的紧迫程度排序，形成改进清单并分配到责任人。

三、评估的发现与架构改进

关于质量属性场景的构建方法。这里需要重点说明，因为场景构建的质量直接决定了评估的质量。我采用“刺激源—刺激—环境—制品—响应—响应度量”六要素来构建每一个场景，缺一不可，尤其是“响应度量”——没有量化度量的场景是无法验证的。

举一个性能场景的例子：刺激源是营业本部的操作员；刺激是在营业高峰期发起预约查询请求；环境是系统正常运行且批量作业同时执行；制品是平台的预约查询接口；响应是接口返回查询结果；响应度量是响应时间不超过 2 秒。

再举一个可用性场景的例子：刺激源是消息投递网关；刺激是对同一业务消息返回超时；环境是系统正常运行；制品是平台的消息推送链路；响应是平台正确处理重试且不产生重复发送；响应度量是重复发送率为 0。

评估发现的敏感点。 敏感点是指对某个质量属性有显著影响的架构决策。本次评估共识别出 12 个，其中三个最为关键。

第一个敏感点是数据库连接池的共享方式。评估中我们通过场景走查发现，若批量作业与在线业务共享连接池，在"批量作业与在线请求并发"这一场景下，性能度量必然不达标。这个发现直接改变了我们后续的设计。

第二个敏感点是依赖版本的管理方式。评估时我们发现部分模块采用了动态版本区间，这意味着构建产物不具备可复现性——同一个提交在不同时间、不同机器上可能解析出不同的依赖版本。这一敏感点影响的是"可修改性"与"可维护性"。

第三个敏感点是用户身份信息的获取位置。平台采用集团统一认证服务进行身份认证，若每次请求都实时向认证服务校验，则认证服务的可用性就直接决定了平台的可用性。

评估发现的权衡点。 权衡点是同时影响多个质量属性、且对一方有利必然对另一方不利的敏感点。本次共识别出 4 个。

权衡点之一是数据存储的一致性策略。我们采用混合存储并以事件驱动实现跨存储同步，这一决策对性能有利（主链路响应时间从 1.8 秒降至 420 毫秒），但对一致性不利（只能达到秒级最终一致）。评估中我们明确记录了这一权衡，并确认了它的适用边界：预约写入等强一致场景仍走事务，只有报表与通知场景才走异步链路。

权衡点之二是计算形态的选择。将数据同步设计为无服务器函数，对弹性与成本有利（平峰期成本下降约 35%），但对延迟与可调试性不利（存在冷启动，且本地难以完整复现云上触发链路）。

权衡点之三是缓存策略。为降低对外部主数据平台的依赖，我们建立了本地只读副本，这对可用性有利，但对数据新鲜度不利——业务侧可能读到数小时前的数据。

权衡点之四是粒度划分。函数按业务域而非按表划分，对一致性与可维护性有利，但对扩容精细度不利——某个低频实体的变更需要整个业务域的函数一起扩缩容。

评估发现的风险点。 风险点是可能导致质量属性不达标、但尚未确定的架构决策。本次共识别出 3 个，其中最紧迫的是"外部系统故障的传导风险"——邮件网关与消息推送服务是外部依赖，若同步调用，外部超时将阻塞平台线程。

四、改进措施的落实

针对评估结论，我推动了三项重点改进。

改进一：读写数据源隔离。 针对连接池共享这一敏感点，我为批处理配置了独立的读写数据源，读取使用只读账号，写入使用写账号，并通过代码包路径隔离两套数据访问接口。同时把事务隔离级别设置为读已提交以减少锁持有时间，把批量游标限制在 200 条以防止内存溢出。改造后，批量作业执行期间在线接口的响应时间稳定在 320 毫秒以内，批量作业自身耗时反而下降约 30%。

改进二：统一依赖版本管理。 针对依赖版本的敏感点，我建立了独立的统一版本管理插件，各模块通过插件接入统一版本源，关键组件使用强制版本策略锁定。改造后，同一提交在任何机器上的构建产物完全一致。这项改进的直接触发因素，是评估后不久发生的一次真实故障——某次依赖升级后 protobuf 公共类型库被解析到候选版本，导致服务启动失败，恰好印证了评估的预判。

改进三：用户信息会话缓存。 针对认证服务单点的敏感点，我把用户身份信息在登录后缓存于服务端会话中，使认证服务短暂不可用时，已登录用户仍可继续操作。这把这个风险的影响范围从"全站不可用"降级为"新用户暂时无法登录"。

五、总结与展望

本次架构评估共识别出 12 个敏感点、4 个权衡点与 3 个风险点，形成改进清单 10 项，其中 8 项已在架构定型前落实，2 项列为后续计划。平台上线后稳定运行约 8 个月，年可用率达到 99.95%，关键接口响应时间、构建可复现性均达到评估时设定的目标。

评估带来的最大收益不是发现了某个具体缺陷，而是**让质量属性的取舍变得显式**。在评估之前，很多决策只存在于架构师的脑中；评估之后，每一项权衡都被记录、被讨论、被确认，参与方对"为什么这样设计"有了共同的理解。这一点在后续的需求变更讨论中体现得尤为明显——当有人提出"能不能把同步改成实时"时，我们不再需要重新论证，只需查阅当时的权衡记录。

目前仍存在两点不足：一是本次评估为一次性活动，尚未形成定期复评机制，架构演化后的质量属性变化缺少持续跟踪；二是评估主要依赖人工走查，缺少量化的架构度量手段，如耦合度、圈复杂度等指标的自动采集。后续计划把架构评估纳入版本迭代流程，每季度复评一次，并引入自动化度量工具辅助评估。

通过本项目，我对架构评估形成了三点认识。第一，评估的价值在于"提前"——在设计阶段发现问题的成本，远低于在上线之后。第二，评估的产出不仅是问题清单，更是共识；让利益相关方共同参与场景构建的过程，本身就是消解分歧的过程。第三，场景的六要素中，"响应度量"最容易被忽略，但它恰恰是评估能否被验证的关键——没有度量的场景只是一句愿望。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

位置	示例值	请替换为
门店数 / 业务人员数	300 余家 / 约 5000 名	真实数量
日均预约量 / 消息量	约 3 万笔 / 约 12 万条	真实数量
敏感点 / 权衡点 / 风险点数量	12 / 4 / 3	如实填写
改进项数量	共 10 项, 落实 8 项	如实填写
主链路响应时间	1.8 秒 → 420 毫秒	真实数据
平峰期成本下降	约 35%	真实数据
改造后响应时间	320 毫秒以内	真实数据
批量作业耗时下降	约 30%	真实数据
稳定运行时长 / 可用率	约 8 个月 / 99.95%	真实指标

本文字数

摘要约 350 字，正文约 2950 字，**合计约 3300 字**（略超，考场可酌情精简第四节）。

可复用要点

- **ATAM 五阶段 + 效用树 + 六要素场景 + 四类要素（敏感点/权衡点/风险点/非风险点）** 是本题的标准骨架，必须背熟
- **场景六要素必须能默写：**刺激源、刺激、环境、制品、响应、响应度量。绝大多数考生会漏掉"响应度量"

- 把项目真实做过的取舍改写成“权衡点”，比空谈方法论有说服力得多（本文列的 4 个权衡点全部来自项目真实决策）
- 「评估最大的收益是让取舍变得显式」是一个高层次的收尾判断
- 若题目指定 SAAM 或 CBAM，把第三节的方法论替换即可，项目素材完全通用

论批处理系统的设计与实现

⚠ 关于本文中的数字 本文的量化数据（作业数量、执行时长、失败率等）均为**示例值**，用于展示论文应有的完整度与写法。架构类数字（表数量、接口数、配置值）已从项目源码核实，可直接使用；业务规模类数字（门店数、用户数、业务量、改善幅度）在项目资料中尚未确定，属示例值，背诵前请替换为你的真实数据。文末附有「待替换数字清单」，便于逐项核对。

试题

论批处理系统的设计与实现

批处理系统是指按照预定规则，在无需人工干预的情况下，对大量数据执行周期性或触发式处理的系统。批处理系统广泛应用于数据同步、报表生成、账单结算、消息批量推送等场景。批处理系统的设计需要重点考虑作业划分与调度、大数据量的处理效率、失败重试与断点续跑、与在线业务的资源隔离以及执行过程的可观测性等问题。

请围绕“批处理系统的设计与实现”论题，依次从以下三个方面进行论述：

1. 简要叙述你参与设计和开发的批处理系统，以及你所担任的工作；
2. 详细论述你在该系统中的批处理设计，包括作业划分、调度策略、容错与重跑机制；
3. 分析你在批处理系统设计与实施中遇到的问题及解决措施。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台中的批处理子系统为例，论述了批处理系统的设计与实现。该子系统承担成本文件生成、外部预约导入、待办状态更新、邮件与通知批量生成、集团主数据同步等作业，需在每日营业开始前完成全部处理，同时不得影响在线业务的响应时间。我在项目中担任系统架构师，负责批处理作业的划分与调度设计、容错与重跑机制设计以及与在线业务的资源隔离方案。针对批量作业与在线业务争抢数据库连接、跨时区环境下日期边界难以把握、异地数据同步缺少可观测性三大挑战，我从资源隔离、参数化基准日、指标主动推送三个维度进行设计：为批处理配置独立的读写数据源并按代码包路径隔离数据访问接口，使批量读取与在线交易在物理层面分离；把执行基准日与取得范围定义为显式作业参数而不依赖系统时钟，使补数重跑的结果完全可复现；由作业主动向监控网关推送执行指标，弥补了短生命周期进程难以被常规监控采集的缺陷。改造后，批量作业执行期间在线接口响应时间稳定在 320 毫秒以内，批量作业自身耗时下降约 30%。

正文

一、项目背景与批处理需求

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务 H.I.S. 集团 300 余家营业门店与约 5000 名业务人员，日均处理预约约 3 万笔、消息与通知约 12 万条。我在项目中担任系统架构师，负责批处理作业的划分与调度设计、容错与重跑机制设计以及与在线业务的资源隔离方案。

平台的批处理需求来自业务的几个刚性环节。

第一是**成本结算**。营业门店的预约在出行前后需要产出成本文件，交由集团财务系统处理。这类作业数据量大、计算密集，且必须在财务结账日前完成。

第二是**外部数据导入**。平台需要定期从外部预约系统拉取预约数据并导入本地，同时从集团主数据平台同步国家、城市、航空公司、货币等主数据。

第三是**状态推进**。预约的待办事项、提醒状态等需要按日推进，例如出行前若干天触发提醒、逾期的待办自动关闭。

第四是**批量通知**。出行前的提醒通知、行程变更的批量告知等，需要在指定时刻批量生成并投递。

综合来看，平台的批处理子系统包含 10 个作业，其中既有每日执行的常规作业，也有按周、按月执行的周期性作业。这些作业的共同约束有两条：一是必须在**次日营业开始前完成**，因为营业开始后数据必须是最新的；二是**不得影响在线业务的响应时间**，因为批量执行时段与部分门店的营业时间存在重叠。

二、批处理架构的设计

作业划分的原则。我在划分作业时遵循三个原则。第一，按业务闭环划分，一个作业完成一件完整的业务事项，而不是按技术步骤拆分——例如“成本文件生成”作业内部包含了数据抽取、计算、文件生成与上传对象存储的完整过程，而不是拆成三个作业由调度器串联。这样做的理由是：拆得过细会使作业间的状态传递成为问题，一旦中间某个作业失败，难以判断重跑应从哪一步开始。第二，按数据边界划分，作业之间不共享数据表，避免一个作业的失败影响另一个作业的输入。第三，按执行频率与时机划分，把日次、周次、月次作业分开调度，使调度配置清晰。

调度策略的设计。平台的批处理不在应用内部定时触发——代码中没有任何内置的定时任务——而是由外部的调度链路驱动。完整的链路是：事件调度器按 cron 表达式触发，启动一个状态机编排流程，由状态机提交批处理计算任务，计算平台拉起容器执行作业，作业完成后进程退出。这条链路的一个关键设计是把“定时”与“计算”分离：调度器只负责在正确的时间点发起，计算平台只负责按需拉起容器，两侧都不需要常驻进程。同时，我把全部 cron 表达式都显式指定为东京时区，避免因平台默认时区与业务时区不一致而在错误的时刻触发——对营业系统而言，在营业时段内跑批处理是不可接受的。

作业的选择通过环境变量指定，作业参数通过容器的启动命令传入。这意味着同一个镜像可以承载全部作业，新增作业不需要新建部署单元，只需增加一条调度配置。

这里还有一个重要的架构决策：**批处理模块不启用 Web 容器**。理由是批处理不需要对外提供 HTTP 服务，若启用 Web 容器，不仅会占用端口与内存，还会使批处理模块具备被外部访问的能力，扩大了安全面。使批处理保持为纯粹的计算进程，既省资源，也降低了风险。

在调度粒度上，我把作业之间的依赖关系交给调度服务管理，而不是在作业代码内部互相调用。这样每个作业都可以独立触发、独立重跑，调度拓扑的调整也不需要修改代码。

容错与重跑机制。这是批处理设计中最关键的部分，我从三个层面进行设计。

第一层是**参数化**。我把执行基准日、取得范围、导入类型等关键参数定义为作业参数显式传入，而不依赖系统时钟取当前日期。这一设计的价值在跨时区部署时尤为突出：平台服务日本本部，而容器运行环境为协调世界时，若依赖系统时钟判断"今天"，在协调世界时的凌晨时段会取到与日本日期不一致的边界，导致漏处理或重复处理。参数化之后，补数时可以指定历史日期重跑，结果与当日运行完全一致；测试时可以固定参数，使结果可预期。

第二层是**幂等性**。批处理的失败重跑是常态，因此每个作业都必须能够安全地重复执行。我的做法是：写入前先按业务键判断是否已处理，已处理则跳过；对于汇总类作业，采用"先清理本次范围、再重新写入"的方式，而不是增量累加，避免重跑导致数据翻倍。对于对外投递类作业（如批量通知），则以投递状态跟踪表记录每一封邮件的投递结果（未投递 / 已投递），重跑时只处理未投递的记录，从根本上杜绝重复发送。

第三层是**可观测性**。批处理属于短生命周期的进程，作业结束后进程即退出，常规的进程存活监控无法采集其运行状态——这是批处理与在线服务在可观测性上最大的差异。为此我引入了指标推送机制：由作业在运行过程中主动把指标推送到 Prometheus 的 PushGateway，并把推送间隔设为 5 秒、关闭动作设为"进程退出前投递"，确保最后一批指标不会随进程退出而丢失。基于这些指标，运维人员可以设置告警规则——例如"作业在预定时刻未推送开始指标"即意味着作业未启动或被阻塞，这是比事后查看日志更及时的发现手段。

三、与在线业务的资源隔离

这一部分是本项目批处理设计中投入最大、收益也最显著的部分。

平台的成本文件生成、外部预约导入、待办状态更新等作业需要从生产库大量读取并回写结果，而在线预约接口同时也在访问同一数据库。在系统上线前的压测中，我们发现夜间批量作业执行期间，在线预约查询接口的响应时间由 300 毫秒劣化至 2.5 秒，已完全超出可用性目标。

经分析，根源是两者共享数据库连接池。批量扫描属于长时间、大数据量的连续查询，会长时间占用连接，而在线接口的请求需要立即获得连接才能响应。在连接池容量有限的情况下，批量的持续占用直接导致在线请求排队。

我采取了四项措施。

第一，**独立数据源**。为批处理配置独立的读写数据源：读取使用只读账号，写入使用写账号，并通过代码包路径隔离两套数据访问接口。这样，批量读取与在线交易在数据库连接层面彻底分离，批量作业无法挤占在线业务的数据源。

第二，**控制事务粒度**。我把事务隔离级别设置为读已提交，减少长事务持有锁的时间；同时把事务边界收敛到独立的服务类中，避免大事务回滚造成连带重放。

第三，**限制批量游标**。我把数据访问层的默认游标大小限制在 200 条，防止海量结果集一次性载入导致内存溢出——这是批处理最常见的故障模式，尤其在数据量随时间增长后才会暴露。需要注意的是，在线侧的同名参数取值为 1000，两侧刻意采用不同的取值：在线查询追求单次取回的效率，批量处理追求内存的可控。

第四，**收敛扫描范围**。对云数据库侧的扫描操作，我把扫描上限显式设置为一个有限值，为可能出现的误用设置硬边界，防止一次疏忽触发全表扫描。

改造后再次压测，批量作业执行期间在线接口的响应时间稳定在 320 毫秒以内；出人意料的是，批量作业自身的耗时反而下降了约 30%——原因是独立的只读数据源使读取不再与在线事务争抢锁资源，而且游标限制让内存回收更及时。

四、实施中遇到的问题及解决措施

问题一：批量作业峰值冲击在线服务。 如上节所述，这是本项目批处理设计中最重要的问题，通过独立数据源、控制事务粒度、限制游标与收敛扫描范围四项措施解决。

问题二：主数据同步失败无从察觉。 平台需要定期从集团主数据平台同步数据。上线初期曾出现主数据更新后业务侧仍在用旧值的情况，而运维人员无从知晓同步是否成功——因为同步作业以批处理方式运行，作业失败后没有任何通知。经排查，根源是缺少可观测手段，而非同步逻辑本身有缺陷。我为此补充了两项措施：一是把同步作业的执行结果纳入监控，作业失败即触发告警；二是让同步作业的基准日期参数化，使补数可以精确指定日期范围重跑。改造后，主数据不一致的问题单数量下降约 90%。

问题三：批处理的重跑导致数据翻倍。 初期有个别汇总类作业采用增量累加的方式写入，一旦因网络抖动重跑，数据就会翻倍且难以察觉。我推动了写入方式的改造，把汇总类作业统一改为“先清理本次范围、再重新写入”，使重跑的结果与首次执行完全一致。这个问题的教训是：批处理的幂等性不能依赖“不会失败”这一假设，而必须在设计时就予以保证。

问题四：跨时区的日期边界。 平台服务日本本部，源数据库运行于日本标准时间，而容器运行环境为协调世界时，开发终端又是另一个时区。三套时区并存时，预约截止时间、邮件发送时刻等跨时区比较极易出错，后果是提前发送或漏发——对旅游业务而言这是不可接受的。我的处理原则是时间口径单一化：存储层统一使用协调世界时；展示层按业务方时区转换；转换动作集中到统一的格式化工具类，禁止在业务代码中散落直接取当前时间的调用。批处理侧则更为彻底——执行基准日与取得范围作为作业参数显式传入，不依赖系统时钟。

五、总结与展望

平台上线至今已稳定运行约 8 个月。批处理方面的成效主要体现在：全部作业均在次日营业开始前完成，未发生因批处理原因导致的营业延迟；批量作业执行期间在线接口响应时间稳定在 320 毫秒以内，批量作业自身耗时下降约 30%；主数据不一致的问题单数量下降约 90%。

目前仍存在三点不足。第一，作业之间的依赖关系由调度服务管理，但依赖关系的可视化程度不足，新成员理解调度拓扑需要阅读大量配置。第二，批处理的重跑目前需人工触发，尚未实现失败作业的自动恢复——这是有意为之的保守选择，因为误判自动重跑可能造成更大的影响，但代价是夜间故障的响应依赖人工值守。第三，批处理与在线业务虽然实现了数据源层面的隔离，但两者仍在同一数据库实例上，尚未实现实例级隔离。

后续计划完善调度拓扑的可视化，引入带条件判断的半自动重跑机制，并在数据量进一步增长时评估实例级隔离方案。

通过本项目，我对批处理系统的设计形成了三点认识。第一，批处理与在线服务的差异不仅在负载特征上，更在**可观测性模型**上——在线服务的健康状态可以通过持续监控获得，而批处理是“一次性事件”，必须由作业主动上报，否则失败的作业就是沉默的失败。第二，**幂等性不是优化项，而是批处理的必备属性**，因为失败重跑在批处理中是常态而非例外。第三，批量作业与在线业务的关系需要主动设计——它们共享同一个数据源是默认状态，而资源隔离是需要额外付出努力才能获得的能力；在项目初期就完成隔离，成本远低于上线后被动改造。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

位置	示例值	请替换为
门店数 / 业务人员数	300 余家 / 约 5000 名	真实数量
日均预约量 / 消息量	约 3 万笔 / 约 12 万条	真实数量
批处理作业数量	10 个	✓ 已核实
压测劣化情况	300ms → 2.5s	真实压测数据
改造后响应时间	320 毫秒以内	建议改用「90 分位 5 秒以内」（有测试报告支撑）
批量作业耗时下降	约 30%	建议改用「4 小时 47 分完成，达标 5 小时以内，吞吐 10 tps」（有实测支撑）
批量游标 / 扫描上限	200 条 / 10000	✓ 已核实
主数据问题单下降	约 90%	真实数据
稳定运行时长	约 8 个月	真实时长

本文字数

摘要约 350 字，正文约 2900 字，合计约 3250 字。

可复用要点

- 「批处理与在线服务的差异不仅在负载，更在可观测性模型」 是本篇最有价值的判断，绝大多数考生写不出来
- 「批量作业与在线业务共享数据源是默认状态，隔离才需要努力」 这句话点破了资源隔离的本质
- 参数化基准日的做法同时服务于"可重跑"与"跨时区正确"两个目标，是一石二鸟的设计点
- 若题目侧重"调度"，把第二节的调度策略展开，补充作业依赖拓扑、触发方式与并发控制
- 若题目侧重"大数据量处理"，把第三节的游标限制、分区读取、扫描边界展开

论软件架构的演化与维护

⚠ 关于本文中的数字 本文的量化数据（遗留模块数量、改造周期、缺陷下降幅度等）均为**示例值**，用于展示论文应有的完整度与写法。**架构类数字（表数量、接口数、配置值）**已从项目源码核实，可直接使用；**业务规模类数字（门店数、用户数、业务量、改善幅度）**在项目资料中尚未确定，属示例值，背诵前请替换为你的真实数据。文末附有「待替换数字清单」，便于逐项核对。

试题

论软件架构的演化与维护

软件架构并非一成不变，随着业务需求的变化、技术栈的更新以及运行环境的发展，软件架构需要持续演化与维护。架构演化既要满足新的业务与技术诉求，又要保证既有功能的稳定运行，因此需要在演进与稳定之间取得平衡。架构演化通常会面临遗留系统难以替换、新旧技术栈长期并存、演化过程中的兼容性保障等问题。

请围绕“软件架构的演化与维护”论题，依次从以下三个方面进行论述：

1. 简要叙述你参与维护和演化的软件系统，以及你所担任的工作；
2. 详细论述你在该系统中如何规划架构演化的路径，以及如何控制演化过程中的风险；
3. 分析你在演化过程中遇到的问题及解决措施。

摘要

本文以我参与建设的 H.I.S. BASE HOME 旅游业务平台为例，论述了软件架构演化与维护的实践。该平台需要长期兼容集团基于旧版 AWS SDK 开发的既有系统，同时要从单体结构向微服务架构演进。我在项目中担任系统架构师，负责演化路径的规划、遗留系统的兼容方案设计以及演化风险的管控。针对遗留技术栈无法一次性重写、新旧版本依赖长期并存、接口契约在演化中容易漂移三大挑战，我采用了三条演化策略：以契约先行稳定接口边界，使内部实现可以自由替换而调用方无感；以渐进替换代替整体重写，按业务域逐个迁移而非一次性切换；以防腐层隔离旧技术栈，把历史包袱约束在有限范围内。同时，我通过统一版本管理插件把第三方依赖这一隐式契约显式化，并建立了以构建可复现性为核心的演化风险管控机制。演化实施后，接口一致性缺陷下降约 85%，构建产物实现完全可复现，因依赖版本漂移导致的运行时故障不再发生。

正文

一、项目背景与演化动因

BASE HOME 平台于 2024 年 3 月启动建设，2025 年 1 月上线运行，服务 H.I.S. 集团 300 余家营业门店与约 5000 名业务人员，日均处理预约约 3 万笔。我在项目中担任系统架构师，负责演化路径的规划、遗留系统的兼容方案设计以及演化风险的管控。

平台的建设并非从零开始，而是在集团既有的系统格局中生长出来的。这决定了我们的架构演化必须面对三个既成事实。

第一是遗留技术栈的长期存在。 集团既有系统中有相当规模的代码基于旧版 AWS SDK 开发，这些系统的业务逻辑仍在正常运行，服务着集团的其他业务线。它们无法被一次性重写，因为重写意味着业务中断、测试成本与回归风险都难以承受。

第二是单体结构的演化压力。 既有系统采用单体部署，其问题在业务量增长后日益明显：任何一种负载的峰值都会波及其他业务，一次变更需要整体回归，任何一个模块的内存泄漏都会拖垮整个进程。平台的伸缩特征差异极大——预约受理是随营业时间波动的交互型负载，成本文件生成是夜间的计算密集型批量负载，数据同步则是持续低频的事件型负载。用单体承载这三类负载，必然导致资源的巨大浪费与相互干扰。

第三是多技术栈协作的现实。 平台需要同时为 Java 内部服务、Go 周边平台与 PHP 既有系统三类调用方提供服务，而这三类调用方的构建工具链、依赖管理方式与发布节奏完全不同。

面对这三个事实，我首先明确了一条原则：**架构演化不是推倒重来，而是在保证业务连续性的前提下逐步替换。**任何需要“停机切换”的方案，都不会被业务方接受，也不应该被架构师提出。

二、演化路径的规划

基于上述原则，我规划了三条并行的演化策略。

策略一：契约先行，稳定接口边界。

这是三条策略中最根本的一条。演化过程中最容易出问题的环节，往往是调用方与实现方被同时改动——一旦双方都需要变更，就必须协同发布，而协同发布的窗口极难把握，且一旦出现问题时无法判断是哪一方的责任。

因此我把平台的接口契约独立到规格模块中，使其不依赖任何业务实现模块。契约按服务定义、请求消息、响应消息、枚举与可复用消息五层组织，由同一份契约驱动 Java、Go、PHP 三语言客户端与 OpenAPI 文档的生成。这样做带来的关键收益是：**接口边界一旦稳定，边界以内的实现就可以自由替换**——内部从单体拆分为服务、从容器改为函数、从一种存储迁移到另一种存储，调用方都无需感知。契约成了演化过程中的“稳定面”，而全部变化都发生在“不稳定面”之内。

在契约本身的演化上，我遵循向后兼容的原则：新增字段而非修改字段，新增接口而非修改接口语义。对于确实需要破坏性变更的场景，则通过并行提供新旧两个版本来过渡，而不是直接替换。

策略二：渐进替换，按业务域逐个迁移。

对于从单体向服务化演进的部分，我没有采用“一次性重写”的方式，而是按业务域逐个迁移。每个业务域的迁移都遵循同样的步骤：先在契约中定义该业务域的接口；再把该业务域的逻辑从单体中抽离，实现为独立服务；随后把调用方逐个切换到新服务，切换期间新旧两条路径并存；最后在确认新路径稳定后，下线单体中的旧实现。

这一策略的核心价值在于**把一次大风险拆成若干次小风险**。每一次迁移的范围有限、影响可控、可独立回滚；即使某次迁移出现问题，也只影响该业务域，不会波及全平台。同时，拨离过程中暴露出的模块耦合问题，会自然地推动我们进一步理清边界——很多在原单体中看似清晰的模块，一旦尝试独立部署，才会发现其与周边模块的隐藏依赖。

对于平台中新增的能力，则直接以服务形态实现，不再往单体中增加代码。这样，单体的规模只减不增，其复杂度随时间推移而收敛，而不是继续膨胀。

策略三：防腐层，隔离旧技术栈。

对于必须与旧版 SDK 共存的场景，我没有要求全平台统一到新版 SDK——这既不现实，也会把旧版本的问题扩散到新代码中。相反，我在新旧之间建立了一层防腐层：把使用旧版 SDK 的代码收敛到明确的边界内——本项目中主要是对象存储与安全令牌服务这两项能力仍在旧版 SDK 上——边界之外的新代码一律使用新版 SDK；跨边界的交互通过平台内部的统一抽象完成，而不是让新版代码直接依赖旧版 API 的类型。

这样做的结果是，历史包袱被约束在有限的范围内，其影响不会扩散到新写的代码。同时，这个边界也为将来的清理提供了明确的范围——当旧系统最终下线时，只需删除防腐层内的代码，而不需要在全平台范围内搜索旧 API 的调用。

三、演化风险的控制

控制一：把隐式契约显式化。

演化过程中出现的一次严重故障，让我重新认识了“契约”的范围。平台初期各模块独立声明依赖版本，部分组件采用了动态版本区间。一次依赖升级后，protobuf 官方公共类型库被解析到一个候选版本，导致 gRPC 服务启动

直接失败。

排查后我意识到，我们虽然统一了业务接口的契约，却忽略了**第三方依赖版本这一隐式契约**——两个模块即使业务接口完全一致，也可能因为底层库版本不同而无法互通。而且这种不一致在编译期完全不可见，只在运行时才暴露。

因此我建立了独立的统一版本管理插件，各模块通过插件接入统一的版本源，关键组件使用强制版本策略锁定。改造后，同一提交在任何机器、任何时间构建出的产物完全一致，构建具备了可复现性。这项改造的价值在演化过程中尤为突出：它使“演化前后的行为差异”可以被明确归因，而不会与“依赖版本漂移导致的差异”混在一起。

控制二：以构建结果界定变更影响。

在多团队协作的演化场景下，一次接口变更的影响范围往往需要人工梳理——哪些调用方会受影响、哪个语言的产物需要重新发布。人工梳理难免遗漏，而遗漏的代价是运行时的集成失败。

由于契约是单一来源且产物由构建流水线自动生成，变更的影响范围可以从构建结果中直接读出。这使变更评审从“依赖经验判断”变为“以构建产物为准”，评审效率与准确性同时提升。

控制三：可回滚优先于可修复。

演化过程中，我要求每一项变更都必须具备回滚路径。具体而言：接口层面保持向后兼容，使新版本可以随时退回旧版本；部署层面通过容器镜像的版本管理，使任何一次发布都可以快速回退到上一个稳定版本；数据层面则通过新增字段而非修改字段的方式，保证新旧代码可以同时读取同一份数据。

这一原则的出发点是：**在演化过程中，“能退回”比“能修好”更重要。**线上的问题往往需要在极短时间内响应，而修复需要定位、编码、测试、发布的完整周期，回滚只需要一次镜像切换。

四、演化过程中遇到的问题及解决措施

问题一：跨语言接口契约的一致性维护。项目初期各端曾各自根据文档维护接口定义，导致同一字段在不同端的命名与类型出现差异，直到联调阶段才被发现。这是典型的演化失控——接口在多方手中各自漂移。我的判断是，根源不是开发人员疏忽，而是**契约缺乏单一来源**这一架构缺陷。因此我把接口定义全部收敛到规格模块，使接口变更必须通过该模块提交，并把三语言产物的生成纳入构建流水线，让不一致问题从“运行时联调发现”提前到“构建期暴露”。改造后，接口一致性缺陷较改造前下降约 85%。

问题二：依赖版本漂移引发的运行时故障。如前所述，protobuf 公共类型库被解析到候选版本，导致 gRPC 服务启动直接失败。这一故障在演化过程中的危害尤其大，因为它发生在新旧代码并存的状态下，容易被误判为演化本身引入的问题。解决方案即统一版本管理插件与强制版本策略。

问题三：旧版与新版 SDK 共存的长期成本。双栈并存的状态在初期是必要的，但随着时间推移，其代价逐渐显现：开发人员需要同时熟悉两套 API，依赖树中存在版本冲突的风险，构建体积与构建时间都有增加。我对此的处理方式是**接受这一成本，但为其设定明确的收敛路径**：把旧版 SDK 的使用范围约束在防腐层内，并禁止新增使用；在每一次业务迭代中，如果某个模块因业务需要而改动，则顺带完成该模块的 SDK 迁移，而不是专门安排一次迁移任务。这种“搭车式迁移”的优点是迁移成本被摊薄到日常迭代中，不需要专门的项目预算与测试周期。需要客观说明的是，目前这一过渡状态尚未终结，长期维护成本依然存在。

问题四：演化中的功能回归。在按业务域逐个迁移的过程中，曾出现过迁移后某个边界场景的行为与原实现不一致的情况。这类问题的根源在于，原实现中的某些行为是历史累积形成的，且并未被文档记录——典型的“隐性知识”。我的应对措施有两条：一是迁移前补充针对该业务域的自动化测试，把可识别的行为固化为测试用例；二是

迁移采用新旧路径并存的方式，通过双跑比对来发现行为差异，而不是直接切换。这样，即使存在未被文档记录的隐性行为，也能在双跑阶段被发现。

五、总结与展望

平台演化实施至今已稳定运行约 8 个月，年可用率达到 99.95%，业务量较原系统提升约 40%。演化方面的成效主要体现在：接口一致性缺陷较改造前下降约 85%；构建产物实现完全可复现，因依赖版本漂移导致的运行时故障不再发生；单体的规模只减不增，复杂度随时间收敛而非膨胀。

目前仍存在三点不足。第一，旧版与新版云 SDK 并存的过渡状态尚未终结，长期维护成本依然存在；第二，契约的破坏性变更尚未实现自动化的兼容性检查，仍依赖人工评审；第三，防腐层内部的历史代码缺少测试覆盖，其行为主要依赖人工理解，这也使得清理工作需要格外谨慎。

后续计划引入契约兼容性校验工具，在构建阶段自动拦截破坏性变更；制定旧版 SDK 的最终收敛时间表；并逐步为防腐层内的代码补充测试，为最终的清理创造条件。

通过本项目，我对架构演化形成了三点认识。第一，演化的核心矛盾是"变"与"不变"的划分——架构师的任务是找到那个可以长期不变的稳定面（在本项目中是接口契约），并让变化都发生在它之后。第二，演化中最危险的不是技术难题，而是**未被记录的隐性行为**；因此"先补测试再迁移"不是保守，而是必需。第三，演化能力的上限取决于构建与部署能力——如果没有可复现的构建、可回滚的发布与自动化生成的契约产物，再好的演化策略也无从落地，这也印证了那句常被引用的话：架构演化的速度，受限于交付流水线的速度。

写作提示（不计入论文字数，背诵时请忽略本节）

待替换数字清单

位置	示例值	请替换为
门店数 / 业务人员数	300 余家 / 约 5000 名	真实数量
日均预约量	约 3 万笔	真实数量
接口一致性缺陷下降	约 85%	真实数据
业务量提升	约 40%	真实数据
稳定运行时长 / 可用率	约 8 个月 / 99.95%	真实指标
遗留 SDK 范围	对象存储 / 安全令牌服务两项	✓ 已核实
迁移的业务域数量	未给出（可补充）	真实数量

本文字数

摘要约 350 字，正文约 2900 字，**合计约 3250 字**。

可复用要点

- 「架构演化不是推倒重来，而是在保证业务连续性前提下逐步替换」 是开篇立场的标准表述
- 三条演化策略**（契约先行稳定边界 / 渐进替换按域迁移 / 防腐层隔离旧栈）中，**绞杀者模式**与**防腐层**是有正式名称的架构模式，写出来能显著提升专业度

- 「能退回比能修好更重要」 是演化风险管控的提炼总结，非常有力
- 「搭车式迁移」 的做法（借业务改动顺带完成技术迁移）是真实可行的工程实践，比"安排专项迁移"更务实
- 「未被记录的隐性行为是演化中最危险的」 是本篇最有洞察力的判断
- 若题目问"遗留系统改造策略"，直接引用第二节的三条策略